

Notes by Cherian George

<https://github.com/cheriangeorge/System-Commands-course>

Week 1 Notes

Terminal Emulators

- Terminal
- Konsole
- xterm
- guake

Command Prompt

- `username@hostname:~$`
 - `~$` is the path

Commands and Flags

- `uname`
 - prints the name, version and other details about the current machine and the operating system running on it
 - the `-a` displays hidden files that have a dot in front of them
- `pwd`
 - Present Working Directory
- `ls` - `a` : all . displays hidden files - `l` : use a long listing format - `i` : print index number of each file (inode) - `s` : shows blocks occupied by each file -
1 : each file name on a separate line
 - output of `ls -l` : `drwxr-xr-x 5 ckg ckg 12288 Nov 25 10:00 Documents` (`d` is file type; `rw-r-xr-x` owner,group,others permissions; `5` no of hard links; `ckg` is owner; `ckg` is group; last modified time stamp; filename)
 - `ls F*` gives a list of all files starting with F
- `rm`
 - remove a file
 - `rm -i` prompts before every removal (it can be set using `alias rm="rm -i"`)
 - works only with write permission
 - use `-d` for removing directories
 - `rm -r mydirectory`
- `mv`
 - move, rename
 - `mv file1 ..` (moves file to parent dir)
 - `mv file1 file1a` (renames file1 to file1a)
- `ps`
 - currently running processes
- `clear`
 - or `ctrl+l`
- `exit`
 - or `ctrl+d`
- `man`
 - get help on any command in linux. eg : `man ls`
 - man sections (1 to 9) eg : `man 1 ls`
 - 1 - Executable programs or shell commands
 - 2 - System calls provided by Kernel
 - 3 - Library calls
 - 4 - Special files usually found in `/dev`
 - 5 - File formats and conversions
 - 6 - Games
 - 7 - Misc : macro packages and conventions
 - 8 - System admin commands
 - 9 - Kernel routines
- `cd`
 - change directory eg `cd ..` - goes to parent directory
 - `cd` without any arguments will take you to the home directory
 - `cd /` takes you to the root folder
 - `cd -` takes you to previous directory
 - `cd ~` takes you to home directory
- `cp`
 - copy command : `cp file1 file2`
- `date`
 - date and time
 - `date -R` gives in RFC 5322 standard (used for email communications)
- `cal`
 - calendar of a month
 - eg : `cal aug 1947`
 - `ncal` gives calendar in flipped orientation
- `free`
 - memory statistics

- use `h` flag to make it human readable
- **groups**
 - groups to which a user belongs
- **file**
 - what type of file
 - `-f` allows you to pass a file in which file names are separated by lines (`ls -l > files.txt; file -f files.txt`)
 - `file *` will give a list of file name and types directly
- **mkdir**
 - create a directory
 - default permissions (umask)
- **touch**
 - used to change the last modified timestamp of a file
 - also used to create empty files
- **chmod**
 - `chmod 777 file.txt`
 - `chmod g-w file.txt` (removes write permissions from the group)
 - `chmod o-x file.txt` (removes executable permission from others)
 - `chmod u-r file.txt` (removes read permission from owner)
- **whoami**
 - prints username
- **less**
 - allows you to read a file page by page
- **ln**
 - used to create a hard link or a symbolic link (symlink) to an existing file or directory
 - `s` flag is used to create a soft link
 - usage: `ln file1 file2 ; ln -s file1 file2`
- **cat**
 - stands for concatenate
 - allows you to view the contents of a single file or multiple files (gets concatenated)

File types

- output of `ls -l` : `drwxrwxrwx` or `lr-x--x--x` (l indicates symbolic link and d indicates directory)
- `-` Regular file
- `d` Directory
- `l` Symbolic link
- `c` Character file (usually found in `/dev` ; typically the terminal)
- `b` Block file (usually found in `/dev` ; typically the hard disk)
- `s` Socket file
- `p` named pipe

Viewing and Adding to files

- `cat` - to view the contents of a file
- writing to a file : `> eg: echo "Hello world" > test.txt`
- appending to a file : `>> eg: echo "Helo world" >> test.txt`

Hard links and Soft links

- inode - An entry in the filesystem table about the location in the storage media
- hard link points to the same inode
- soft link points to a hard link
- hard link must be on the same partition while soft link can point to a file at a totally different geographical location.
- inode is metadata for the file . eg : size ,permissions,blocks etc.
- `ls -li <name>`
- `ln` and `ln -s` is used for creating hard links and soft links
- inode is unique for every file : if there are multiple entries of inode then it means that they are all hard links
 - if there is a dir level1 with inode = 18874686
 - when you `cd` into that dir . will also have inode = 18874686
 - if i make a dir level2 inside level1 and then `cd` into level2 .. will have inode = 18874686 (no of hard links will increase by 1)
 - as number of sub directories increases the number of hardlinks also keeps increasing
- users cannot create hard links for directories (level1 to level2 and level2 to level1 will create a back and forth)

Permissions

- Files and directories do not inherit the parent directory permissions
- `rw-rw-rw-` (777)
 - 7 `rw-`
 - 6 `rw-`
 - 5 `r-x`
 - 4 `r--`
 - 3 `-wx`
 - 2 `-w-`
 - 1 `--x`
- `rw-rw-rw-` : Owner Group Others
- only owners can change permissions of a file
- Execute permission is required on a directory to `cd` into it (Even `ls` and `touch` to a dir will not work)
- If you want to access a file, all its parent directories should have `x` permission. This works even without `r` and `w` permissions if you know the path.

- r and w permissions along with x is required to ls a directory or touch a file into a directory
- Removing a file works only if it has write permission

Linux Virtual Machine

- ##### ISO
 - image of Linux OS (Ubuntu 20.04 LTS for x86_64 platform)
- ##### Hypervisor
 - (eg: Oracle VirtualBox or VMWare Workstation Player)
 - A Hypervisor creates and runs virtual machines
 - It allows running multiple operating systems while sharing hardware resources

Command Line Environments

- Cloud - replit and cocalc
- Phone - Termux by Fredrick Fornwall

File System of Linux OS

- Filesystem Hierarchy Standard FHS 3.0 (June 03, 2015) (refspecs.linuxfoundation.org/fhs.shtml)
- / is root directory and field separator or delimiter for sub-directories
- . references the current directory (. is a special file in every directory)
- .. references the parent directory (.. is a special file in every directory)
- Path for traversal can be absolute or relative
- boot directory is where the kernel is located
- /usr/bin contains commands that we will use
- /bin - essential command binaries
- /boot static files of the bootloader
- /dev device files (different character in long format of file listing 'c' instead of 'l' or 'd'. 'c' indicates character file - means you can read from it character by character. if first character is 'b' they are block devices typically hdds - the block devices are made available as files.)
- /etc Host specific system configuration (.conf files)
- /lib Essential shared libraries and kernel modules (Typically contain files with version number at the end)
- /media mount points for removable devices
- /mnt mount points
- /opt add on application software packages
- /run Data relevant to running processes
- /sbin essential system binaries
- /srv data for services
- /tmp temporary files (normally flushed when system is rebooted)
- /usr secondary hierarchy
 - /usr/bin : user commands
 - /usr/lib : libraries
 - /usr/local : local hierarchy
 - /usr/sbin : non vital system binaries
 - /usr/share : architecture dependent data
 - /usr/include : header files included by c programs
 - /usr/src : source code
- /var variable data (/var/log contains logs for various services)
 - /var/cache : Application cache data
 - /var/lib : Variable state information
 - /var/local : variable data for /usr/local
 - /var/lock : lock files
 - /var/log : log files and directories
 - /var/run : data relevant to running processes
 - /var/tmp : temporary files preserved between reboots

	Shareable	Unsharable
static	/usr and /opt	/etc and /boot
variable	/var/mail	/var/run and /var/lock

Week 2 Notes

- Multiple uses of / is as good as one
 - ie: cd usr/////bin will take you to usr/bin
- The root folder / is its own parent
 - ie: if you do cd .. within the root directory you stay in the same directory.
- Options / Flags can be written in multiple combinations
 - ls -l level1 -di
 - ls -d level1 -il
 - ls level1 -ldi
 - ls -ldi level1

- long formats for options are also available
- `ls -a` is equivalent to `ls --all`

Commands

- `ls`
 - R flag lists all subdirectories recursively
 - Passing directory name to `ls` shows what is within that directory. ie : `ls -l level1`
 - d flag displays details of a folder without traversing inside it. it : `ls -ld level1`
 -
- `ll`
 - a shortcut for the `ls -la` command
- `which`
 - `which command` will show the location of the command
 - `which less` will show `/usr/bin/less`
- `whatis`
 - gives a brief description of the command
- `alias`
 - give a nickname to a frequently used command
 - usage : `alias ll = 'ls -l'`
 - Just typing `alias` will show a list of aliases
 - `alias date = 'date -R'`
 - If the command is executed by typing the whole path eg : `/usr/bin/date` the alias is not invoked. (`cd /usr/bin` and `./date`)
 - An alias can be escaped by prefixing a `\` ie: `\date`
- `unalias`
 - used to remove an alias
- `rmdir`
 - removes an empty directory
- `ps`
 - displays current processes
 - `ps --forest` - which process has launched which child process.
 - `ps -f` - displays parent process id
 - `ps -ef` - all the processes running in the operating system now
 - PID is the process ID , PPID is the parent process ID.
 - PID 1 is `/sbin/init`
- `bc` - bench calculator
 - exit using `Ctrl + D`

Commands to know contents of a text file

- `less`
 - displays the content in one screen
 - `ls -l /usr/bin/less` shows that the command takes 180KB
- `wc`
 - prints newline,word and byte counts for the file
 - the `-l` flag shows just the number of lines
- `head`
 - head profile displays the first ten lines
 - use `-n` flag to specify the number of lines
- `tail`
 - tail profile displays the last ten lines
 - use `-n` flag to specify number of lines to be displayed
- `cat`
 - in `/etc` , cat profile would just dump contents on the screen without any further prompts.
 - disadvantages : cant move back and forth to view page by page, can't come out half way through.
 - if the file is very long cat is not the best way to look at the content.
- `more`
 - similar to `less`. Allows page by page viewing
 - `ls -l /usr/bin/more` shows that the command takes 43KB

Knowing more commands

- `man`
- `which`
- `apropos`
 - For a keyword it shows you all the commands which have that keyword in the description
 - Used to discover new commands
 - If you type `ls -l /usr/bin/apropos` you see that it is a symbolic link to `whatis`, but the outputs are different : Why?
 - Reason : In Linux every executable will know in what name it has been invoked - can have different behaviour depending on the name that invoked it.
 - It also has the same output as `man -k` : Searching for a keyword
- `info`
 - Allows browsing through commands using the cursor
 - Can go back using `<` or 'shift+',
- `whatis`
- `help`
 - displays keywords reserved for the shell being run
- `type`

- displays what type of command it is
- type type shows that it is a 'shell built in' being offered from the shell and not the os
- type ls shows that it is aliased with some option. which ls shows that it is coming from os because there is an executable available.

Multiple Arguments

- ##### Recap : Arguments and Options
- Options are enhanced features of the command
- Arguments are specific names of files or directories
- Second argument behaviour and interpretation of last argument should be seen in the man pages
- Recursion is assumed for mv and not cp
- recursion is assumed for some commands and should be explicitly stated in others
- For copy command recursion is not assumed
- cp dir1 dir2 need not work. dir1 has 2 files in it.
- cp -r dir1 dir2 works - recursion is specified explicitly.
- mv dir1 dir3 works - it just renames the directory.
- touch file1 file2 file3 creates all 3 files in one go with identical timestamp.

Links (Hard Links and Soft Links)

- Can determine whether a link is HL or SL by looking at the Inode numbers
 - Hard links will have the same inode numbers
 - Soft Link will have different inode numbers
 - If you delete a certain file using the rm command (rm unlinks the file from the filesystem. the data is still at the memory location. shred for permanent deletion)
 - Its hard link will still give you access to the original file data.
 - Its soft link will not work
- ln -s source destination to create symbolic link. ln -s file1 file2
 - file2 is a separate inode entry but it is just a shortcut to file1
 - file2 has only 1 hardlink.
- ln source destination to create a hard link. ln file1 file3
 - file1 and file 3 have the same inode number - They are basically the same file.
 - file1 and file3 have 2 hard links when we do ls -li
- You can create a Soft Link ln -s ../dir/filex fileSL but creating a hard link using ln ../dir/filex fileHL will not work.
 - the first/source-file parameter is interpreted in the case of hard link creation and not in soft link creation
 - In the above example, assume that ../dir/filex does not exist.
 - soft links useful in version control systems

File Sizes

- ls -s
 - file size appears in the first column
- stat
 - in /usr/bin we look at stat znew
 - Gives information about the size, how many blocks are being occupied
 - Here the size is little more than 4kb
 - stat zmore shows that it takes less than one block
- du
 - in /usr/bin we look at du znew or du -h znew
 - Gives information about the size
 - Here the size is displayed as 8.0KB since there is a block overflow.
 - This means that files that are smaller than the block size will actually take up a whole block
 - du -h zmore shows that it occupies one block - around 4.0K
- Role of block size
 - explained in stat and du

In-Memory File Systems

- /proc
 - Is an older system
 - ls -l will display several zero-size files, even though we can read content from them.
 - These are only a representation and not real files on the HDD.
 - less cpuinfo - information about the cpu
 - cat version - information about the OS. Also accessible using uname -a
 - cat meminfo - information about the memory - also free -h
 - cat partitions - information about the partitions - also df -h
 - The kcore file appears to take huge space - Shows maximum virtual memory that the current linux os is able to handle. 2^47 or 140 TB
- /sys
 - Used from Kernel v2.6 onwards, however information about various processes that are running are still stored in the /proc directory itself.
 - Much more well organised than /proc
 - eg: sys/bus/usb/devices/1-1 points to a specific usb device.
- These are directories that are visible in the root folder. They are not on the disk but only in the memory.
- Important system information can be viewed from these directories in a read-only manner.

Shell Variables

- Makes it possible to communicate between 2 processes very efficiently. Need not write and read the filesystem.
- Security Concern : Some information that you write to the filesystem may be visible to other processes.
- Shell variables are available only within the shell or its child processes.

- `echo` prints strings to screen
 - uses space as a delimiter so multiple spaces between words are ignored. For multiple spaces, enclose the string in quotes.
 - can print a multi-line string by using double quotes and not closing it
 - **** Difference between ' and " ****
 - `echo $USERNAME` and `echo "$USERNAME"` give the same result but `echo '$USERNAME'` is not interpreted to give the value of the shell variable.
 - **** Escaping to prevent interpretation ****
 - `echo "username is $USERNAME and host name is \ $HOSTNAME"`
 - Escaping is useful when you want to pass on the information to a child shell, without it being interpreted by the shell launching it.
- `echo $HOME` prints values of variables
 - By convention every shell variable starts with a Dollar
- **Commonly used shell variables**
 - `$USERNAME` eg: `echo "User logged into system now is : $USERNAME"`
 - `$HOME`
 - `$HOSTNAME`
 - `$PWD`
 - `$PATH` - variable contains a list of directories which will be searched when you type a command. When ever you type a command the system scans these paths from left to right to see if the command is in the directory.
- Commands like `printenv`, `env`, `set` to see variables that are already defined
 - `printenv` displays all the shell variables defined in the shell that you are running.
 - `env` gives the same output
 - `set` displays some functions defined to interpret what you are typing on the command line.
- **Special Shell Variables**
 - `$0` : name of the shell eg `bash` or `ksh`
 - `$$` : process ID of the shell
 - `$?` : return code of previously run program
 - `$-` : flags set in the bash shell . The man page for bash shows the meaning of the flags.
- **Process Control** `echo $$`
 - use of `&` to run a job in the background
 - `fg` - bring process to foreground
 - `coproc` - run a command while also being able to use the shell
 - `jobs` - list programs running in the background
 - `top` - See programs that are hogging the CPU or memory (refreshed every second)
 - `kill` - kill process owned by you
- **Program Exit Codes** `echo $?`
 - exit code always has a value between *0 and 255*
 - `0` : Success
 - `1` : Failure
 - `2` : Misuse (insufficient permissions)
 - `126` : command cannot be executed (usually due to insufficient permissions to execute a file)
 - `127` : command not found (usually due to command typos)
 - `130` : processes killed using `ctrl+c`
 - `137` : processes killed using `kill -9 <pid>`
 - If the exit code is more than 256 then the `exitcode%256` will be reported as the exit code
 - `exit 0` or `exit 1` or `exit <n>` exits with exit code `n`
 - Used when there are command dependencies (ie: run second command only if first command completes successfully)
- **Flags set in bash** `echo $-`
 - `h` : locate hash commands
 - `B` : braceexpansion enabled
 - `i` : interactive mode
 - `m` : job control enabled (can be taken to `bg` or `fg`)
 - `H` : !style history substitution enabled
 - `s` : commands are read from stdin
 - `c` : commands are read from arguments

Linux Process Management

- `sleep` command to create processes
 - usage: `sleep 3` for 3 seconds
- If you have a command running in the Foreground for a long time but you need to write something else on the command line :
 - kill the process
 - suspend the process
 - run it in the background `coproc sleep 10` - When complete it gives a message.
- `coproc` is a shell keyword. No manual entry for it.
 - To learn more about a shell key word use `help coproc`
 - a running background process can be killed by process id (use: `ps --forest` to find PID and `kill -9 <pid>`)
- A command followed by an `&` means that it is being assigned to the background
 - Executing the command `fg` will bring it back to foreground
- `jobs` is a shell builtin - it lists active jobs in the current shell
- `top` shows processes taking up maximum cpu and memory. Exit gracefully by pressing `Q`
- `Ctrl + z` suspends a process.
 - Suspended processes can be seen with `jobs`
 - Can be brought back to foreground using `fg` command
- `Ctrl + c` kills a process
- `fg` is a shell builtin
- `bash -c "echo \ $-"` creates a child shell, gets the value of `echo $-`, gives the output to the parent shell
 - `bash -c "echo \ $- ; ps --forest;"` - multiple commands separated by ;
 - `bash -c "echo \ $$ ; ps --forest ; exit 300"` : custom error code mod 256 = 44
- `history` displays a list of commands that have been run on that computer
 - `!n` executes command line no `n` displayed by `history`

- o useful for repeating long commands
 - o The `H` flag in bash means the history is being recorded
- Brace expansion option `B`
 - o if you type `echo {a..z}` character in the ASCII sequence will be expanded.
 - o In combination `echo {a..d}{a..d}` will display all possible combinations of the 2 alphabets.
 - o `*` expands to all the files in the current directory
 - o `echo D*` lists all the files beginning with D.
 - o Examples :
 - o `mkdir {1..12}{A..E}` or `rmdir {1..12}{A..E}` or `touch {1..12}{A..E}/{1..40}`
- `;` acts as a separator between individual commands eg: `echo hello ; ls`

REPLIT CODE WITH US

[Link to Replit](#) - `date -d "2024-04-01" +%A` - Day of the week for given date - `file --mime-type somefile` - mime type of a given file - `mkdir {1..12}{A..E}` - `rmdir {1..12}{A..E}` - `touch {1..12}{A..E}/{1..40}` - `lscpu | grep -i "model name" | cut -d ":" -f "2"`

Week 3 Notes

Combining Commands and Files

- Executing Multiple Commands
 - o `command1; command2; command3;`
 - o Each command will be executed one after the other.
 - o `command1 && command2`
 - o `command2` will be executed only if `command 1` succeeds
 - o If the return code is 0 it is true and if it is greater than 0 it is false
 - o `ls && date -Q && wc -l /etc/profile` will display the dir listing followed by error that `-Q` is invalid; `wc` is not executed.
 - o `command1 || command2`
 - o `command2` will not be executed if `command1` succeeds
 - o `ls /blah || date` will display current date after "No such file or directory"
 - o `ls || date` will display just the directory listing
 - o `command2` is like a Plan B if `command1` doesn't succeed.
 - o Example `ls /blah ; date ; wc -l /etc/profile ;`
 - o If we use parenthesis ie `(ls /blah ; date ; wc -l /etc/profile ;)` the command gets executed in a subshell and is returned back to the shell we are using.
 - o We can use `echo $BASH_SUBSHELL` to return an integer which tells us at what level of execution we are.
 - o `(echo $BASH_SUBSHELL)` will report a value of 1
 - o `(ls; (date; echo $BASH_SUBSHELL))` will report a value of 2
 - o Launching too many subshells could be expensive computationally.
- File Descriptors
 - o Every command in linux has 3 file descriptors - `stdin` (0), `stdout` (1), `stderr` (2).
 - o `stdin` is a pointer to a stream that is coming from the keyboard or user input
 - o `stdout` or `stderr` usually points to the screen where the display or output is made.
 - o the three pointers are looking at only the stream of characters.
 - o they can be directed to a file or a command, or the default behaviour can be left as it is.
 - o Combining a command and a file
 - o `command > file1`
 - `stdout` is redirected to `file1`
 - `file1` will be created if it does not exist
 - if `file1` exists, its contents will be overwritten
 - example: `ls -l /usr/bin > file1` - displays no output on the screen because there is no error
 - `ls -l /blah > file1` - displays an error. `file1` is overwritten and is now 0 Bytes.
 - `hwinfo > hwinfo.txt`
 - trying this command in a folder where there is no `w` permissions will generate an error
 - The `cat` command tries to read from the provided file name if not given it tries to read from `stdin` (keyboard)
 - `cat > file1` will allow you to type content. The feature could be used to create text files on the command line. You can come out using the `Ctrl+D` option.
 - `cat file1` displays the content of `file1`
 - `cat` takes input from the keyboard and displays it on the screen (line by line; when you press enter) - Finish by pressing `Ctrl + D` to signify end of file.
 - o `command >> file1`
 - contents will be appended to `file1`
 - new `file1` will be created if it does not exist.
 - Example: `date >> file2 ; wc -l /etc/profile >> file2 ; file /usr/bin/znew >> file2 ;`
 - `cat >> file1` to append text to a file from command line. Come out using `Ctrl + D`

Redirections

- combining command and file (continued ..)
 - o (contd..)
 - o `command 2> file1`
 - redirects `stderr` to `file1`
 - `file1`, if it exists, will be overwritten.
 - `file1` will be created if it does not exist.
 - Example `ls $HOME /blah 2> error.txt`
 - o `command > file1 2> file2`

- `stdout` is redirected to `file1`
 - `stderr` is redirected to `file2`
 - Contents of `file1` and `file2` will be overwritten.
 - The output is in one file and the errors are in another file.
 - Example: `ls $HOME /blah > output.txt 2> error.txt`
 - `ls -R /etc > output.txt 2> error.txt` - permission related errors in `error.txt`
- `command < file1`
 - `stdin` is redirected - a command expecting input from the keyboard could take the input from a file.
 - Example: `wc /etc/profile` behaves similar to `wc < /etc/profile`
- `command > file1 2>&1`
 - command output will be redirected to `file1`
 - `2>` indicates `stderr` and that is being redirected to `&1` (first stream) which is `stdout`
 - contents of `file1` will be overwritten
 - Example: `ls $HOME /blah > file1` output alone is sent to `file1`. Error on screen
 - Example: `ls $HOME /blah > file1 2>&1` output and error is sent to `file1`.
- `command1 | command2` Pipe
 - `stdout` output of command 1 is sent to `stdin` of command2 as input
 - Example `ls /usr/bin | wc -l`
- `command1 | command2 > file1`
 - `command1` and `command2` are combined and the `stdout` of `command2` is sent to `file1`. Errors are still shown on the screen.
 - Example `ls /usr/bin | wc -l > file1` - `file1` has the number of lines counted by `wc`
- `command > file1 2> /dev/null`
 - `/dev/null` file - A sink for output to be discarded. Like a "black hole"
 - We normally don't do anything with the `/dev` folder as there are sensitive system files there.
 - If you are confident that the script is running well and you do not want to display any error on the screen, you can redirect the `stderr` to `/dev/null`
 - `stderr` is redirected to `/dev/null`
 - Example: `ls $HOME /blah > file1 2> /dev/null`
 - Example: `ls -R /etc > file1 2> /dev/null` - `file1` contains the output except errors
- `command1 | tee file1`
 - Used in situations where you want to have a copy of the output in a file as well as on the screen.
 - The `tee` command reads from `stdin` and writes to `stdout` and `file/s`.
 - Example: `ls $HOME | tee file1` also `ls $HOME | tee file1 file2` for creating multiple copies
 - `diff file1 file2` compares files line by line
 - no output if the files are identical
 - Example: `ls $HOME /blah | tee file1 file2 | wc -l` - Here `tee` keeps copy of output in a file and also sends output to `wc -l` for further processing.
 - Example: `ls $HOME /blah 2> /dev/null | tee file1 file2 | wc -l` to suppress errors. Note location of `2>` is since the error is generated there.

Shell Variables - Part 1

- Creation, inspection, modification, lists
- Creating a variable
 - `myvar="value string"`
 - `myvar` can't start with a number, but you can mix alphanumeric and `_`
 - No space around the `=`
 - `"value string"` is the number, string or `command`. Output of a command can be assigned to `myvar` by enclosing the command in back-ticks.
- Exporting a variable
 - `export myvar="value string"` or
 - `myvar="value string" ; export myvar`
 - This makes the value of the variable available to a shell that is spawned by the current shell.
- Using variable values
 - `echo $myvar`
 - `echo ${myvar}`
 - can manipulate the value of the variable by inserting some commands within the braces.
 - `echo "${myvar}_something"`
- Removing a variable
 - `unset myvar`
 - Removing value of a variable `myvar=`
- Test if a variable is set
 - `[[ -v myvar ]] ; echo $?`
 - 0 : success (variable `myvar` is set)
 - 1 : failure (variable `myvar` is not set)
 - `[[ -z ${myvar+x} ]] ; echo $?` (the `x` can be any string)
 - 0 : success (variable `myvar` is not set)
 - 1 : failure (variable `myvar` is set)
- Substitute default value
 - If the variable `myvar` is not set, use "default" as its default value
 - `echo ${myvar:-"default"}`
 - if `myvar` is set display its value
 - else display "default"
- Set default value
 - If the variable `myvar` is not set then set "default" as its value
 - `echo ${myvar:= "default"}`
 - if `myvar` is set display its value
 - else set "default" as its value and display its new value
- Reset value if variable is set
 - If the variable `myvar` is set, then set "default" as its value

- `echo ${myvar:+"default"}`
 - if `myvar` is set, then set "default" as its value and display the new value
 - else display null
- List of variable names
 - `echo ${!H*}`
 - displays the list of names of shell variables that start with H
- Length of string value
 - `echo ${#myvar}`
 - Display length of the string value of the variable `myvar`
 - if `myvar` is not set then display 0
- Slice of a string value
 - `echo ${myvar:5:4}` (5 is the offset and 4 is the slice length)
 - Display 4 characters of the string value of the variable `myvar` after skipping first 5 characters.
 - if the slice length is larger than the length of the string then only what is available in the string will be displayed.
 - the offset can also be negative. However you need to provide a *space* after the `:` to avoid confusion with the earlier usage of the `:` - symbol. The offset would come from the right hand side of the string.
- Remove matching pattern
 - `echo ${myvar#pattern}` - matches once
 - `echo ${myvar##pattern}` - matches maximum possible
 - Whatever is matching the pattern will be removed and the rest of it will be displayed on the screen.
- Keep matching pattern
 - `echo ${myvar%pattern}` - matches once
 - `echo ${myvar%%pattern}` - matches maximum possible
- Replace matching pattern
 - `echo ${myvar/pattern/string}` - match once and replace with string
 - `echo ${myvar//pattern/string}` - match max possible and replace with string
- Replace matching pattern by location
 - `echo ${myvar/#pattern/string}` - match at beginning and replace with string
 - `echo ${myvar/%pattern/string}` - match at the end and replace with string
- Changing case
 - `echo ${myvar,}` - Change the first character to lower case.
 - `echo ${myvar,,}` - Change all characters to lower case.
 - `echo ${myvar^}` - Change first character to uppercase
 - `echo ${myvar^^}` - Change all characters to upper case
 - The original value of the variable is not changed. Only the display will be modified as the trigger commands are within braces.
- Restricting value types
 - `declare -i myvar` - only integers assigned
 - `declare -l myvar` - Only lower case chars assigned
 - `declare -u myvar` - Only upper case chars assigned
 - `declare -r myvar` - Variable is read only
 - Once a variable is set as read only you may have to restart the bash to be able to set it
- Removing restrictions
 - `declare +i myvar` - integer restriction removed
 - `declare +l myvar` - lower case chars restriction removed
 - `declare +u myvar` - upper case chars restriction removed
 - `declare +r myvar` - *Can't do once it is read-only*
- Indexed arrays
 - `declare -a arr`
 - Declare `arr` as an indexed array
 - `$arr[0]="value"`
 - Set value of element with index 0 in the array
 - `echo ${arr[0]}`
 - Value of element with index 0 in the array
 - `echo ${#arr[@]}`
 - Number of elements in the array. The `@` symbol is a wild character to run through all the elements in the array
 - `echo ${!arr[@]}`
 - Display all indices used
 - `echo ${arr[@]}`
 - Display values of all elements of the array
 - `unset 'arr[2]'`
 - Delete element with index 2 in the array
 - `arr+=("value")`
 - Append an element with a value to the end of the array
- Associative arrays
 - `declare -A hash`
 - declare `hash` as an associative array
 - `${hash["a"]}="value"`
 - set the value of element with index a in the array
 - `echo ${hash["a"]}`
 - value of element with index a in the array
 - `echo ${#hash[@]}`
 - number of elements in the array
 - `echo ${!hash[@]}`
 - display all indices used
 - `echo ${hash[@]}`
 - display values of all elements of the array
 - `unset 'hash["a"]'`
 - delete an element with index a in the array
 - Can do everything in the indexed array except append because there is nothing called the end of the array as there is no sequence for the elements of a hash

- Examples

- `true` always returns exit code 0
- `false` always returns exit code 1 (Check with `echo $?`)
- To check whether a variable is present
- `[[ -v myvar ]] ; echo $?` returns 1 if the variable is not present in the memory
- `[[ -z ${myvar+x} ]] ; echo $?` returns 0 if variable is not present and 1 if it is present. `x` is a string that will be used as a replacement if the variable was not present.
- Use of Braces
- `myvar=FileName`
- `echo $myvar`
- `echo "${myvar}.txt"` prints `FileName.txt`
- `echo "$myvar_txt"` does not print anything as the variable `myvar_txt` does not exist
- `echo "${myvar}_txt"` prints `Filename_txt`
- Braces are useful in stating clearly the name of the variable.
- Can also be used outside quotes `echo ${myvar}`
- Does the variable we have created get passed on to the shell or any other program created within the shell
- `myvar=3.14 ; echo $myvar`
- `bash` one more level of `bash`
- `ps --forest` to show that we are one level below
- `echo $myvar` not present
- Use `export myvar=3.14` to ensure this variable is available to all spawned sub shells.
- Change value of variable within the child shell
- modification of value is not reflected in the value of the variable in the parent shell
- even if you do export of the variable within the child shell it will not change the value within the parent shell.
- Use of back-ticks
- `mydate=`date`` value of `mydate` will be output of `date`.
- `mydate=`echo Sunday that is today` ; echo $mydate`
- Manipulations for variables within the shell environment
- We would like to have `echo` display a default value if variable is not available
- `echo ${myvar:-hello}` the `-` indicates if the value is not present what is the display value
- `echo ${myvar:-"myvar is not set"}`
- Set the value if it was not set already
- `echo ${myvar:=hello}` if absent / not set then set it to the value after `=`
- If it is present it will not change
- `echo ${myvar:? "myvar is not set"}` displays a little more information and a debug message. `bash: myvar: myvar is not set`
- Unset the value of a variable using `unset myvar`
- `echo ${myvar:+HELLO}` displays the message if the variable is present
- Inspecting all the variables in the shell environment
- `printenv`
- `env`
- `echo ${!H*}` displays the names of variables beginning with 'H' - `!` indicates names of the variables instead of value.
- Counting characters
- `mydate=`date`` stores the output of the `date` command in `mydate`
- `echo ${#mydate}` prints the length of the value present in `mydate`.
- length of a non-existing variable is zero
- Features of using colon : within braces {}
- Extracting part of a string from the value of a particular variable.
 - `echo ${mydate:6:10}`
 - `echo ${myvar:3:3}` will print `def` for `myvar=abcdefg` ie: 3 characters after the offset (position 3)
- Using negative offset
 - `echo ${myvar: -3:3}` and `echo ${myvar: -3:4}` will print `efg` for `myvar=abcdefg`
 - note `-` is to be preceded with a blank to avoid confusion
 - asking for more characters, will print just what is available
 - `echo ${myvar: -3:2}` will print `ef` for `myvar=abcdefg`
- Extracting a portion of the date
 - Output of `date` is `Tuesday 25 January 2022 09:10:20 PM IST`
 - Output of `date +%d %B %Y` is `25 January 2022`
 - if `mydate=`date`` then `echo ${mydate:8:16}` will also print `25 January 2022`
- Extracting patterns from a string
 - `myvar=filename.txt.jpg`
 - `echo ${myvar#*.}` minimal matching displays `txt.jpg`
 - `myvar=filename.somethingelse.jpeg`
 - `echo ${myvar##*.}` maximal matching displays `jpeg`
 - `echo ${myvar%*.}` displays `filename.somethingelse`
 - the `%` is used to indicate what has not been matched. (minimal)
 - `echo ${myvar%*.*}` displays `filename`
 - the `%` is used to indicate what has not been matched. (maximal)
 - Can be combined `echo ${myvar%*.*}.${myvar##*.*}` to get `filename.jpeg`
 - Replacing what has been matched
 - Pattern matching in Linux usually goes with a pair of forward slashes.
 - Convert all `e` to `E` in a string
 - `echo ${myvar/e/E}` replaces only the first occurrence of `e`
 - `echo ${myvar//e/E}` replaces all occurrences of `e`
 - Replace characters at the beginning of a string
 - `echo ${myvar/#f/F}` replaces the occurrence of `f` in the beginning of the string with `F`. The `#` indicates the beginning of the string
 - Replace characters at the end of a string
 - `echo ${myvar/%g/G}` replaces the occurrence of `g` at the end of the string with `G`. The `%` indicates the end of the string.
 - Replace `jpeg` with `jpg`, only if it is at the end of a string
 - `echo ${myvar/%jpeg/jpg}`
 - Modifying and storing it in a variable

- `myvar1=`echo ${myvar//jpeg/jpg}``
- Generic command to remove day from date
- `echo ${mydate#*day}`
- Upper case to lower case and vice-versa
- `echo ${mydate,}` changes first character to lowercase
- `echo ${mydate,,}` converts all characters to lowercase
- `echo ${mydate^}` changes first character to uppercase
- `echo ${mydate^^}` changes all characters to uppercase
- Restricting values that can be assigned to shell variables using `declare`
- `declare` is a shell builtin
- `+` to **unset** a restriction and `-` to **set** it (Note : counterintuitive)
- `-a` for indexed arrays (need not be ordered indexes)
- `-A` for associative arrays (dictionaries)
- `-i` for integers
- `-u` for uppercase conversion on assignment
- Integer restriction
 - `declare -i mynum`
 - `mynum=10` will assign 10 to mynum
 - `mynum=hello` will assign 0 to mynum
- lowercase restriction
 - `declare -l myvar`
 - `myvar=hello` assigns hello to myvar
 - `myvar=BELLOW` converts BELLOW to lowercase and assigns it to myvar.
- removing a restriction
 - `declare +l myvar`
 - the value is still contained after removing the restriction but you can now store upper case characters as well
- declaring a read-only variable
 - `declare -r myvar`
 - once a variable has been set as read only, you cannot change its value and you cannot remove the read-only restriction using `+r`
 - `declare +r myvar` gives the error `bash: declare: myvar: readonly variable`
- Arrays
- `declare -a arr`
- `arr[0]=Sunday`
- `arr[1]=Monday`
- `echo ${arr[0]}`
- `echo ${arr[1]}`
- `echo ${#arr[@]}` gives number of elements in the array
- `echo ${arr[@]}` displays all values
- `echo ${!arr[@]}` displays the indices
- You can have any index without filling up intermediate indices. Indices are not necessarily contiguous.
- `arr[100]=Friday` is also valid
- Removing an element from an array = `unset 'arr[100]'`
- Appending to an array `arr+=(Tuesday)`
- Populating an array in one go `arr=(Sunday Monday Tuesday)` . The indices are sequential
- Associative Arrays / Hashes
- `declare -A hash`
- `hash[0]="Amal"`
- `hash["mm12b001"]="Charlie"`
- `echo ${!hash[@]}` to get indices
- `echo ${hash["mm12b001"]}`
- File names in a shell variable
- `myfiles=(`ls`)`
- `echo ${myfiles[@]}`

Week 4 Notes

Software Management

- Using Package Management Systems
 - Tools for installing, updating, removing and managing software
 - Install new / updated software across network
 - Package - File look up, both ways
 - Which files are given by a particular package and which package contains a given file
 - Database of packages on the system including versions (compatibility and requirements)
 - Dependency checking
 - Signature verification tools (to check authenticity of source of the software)
 - Tools for building packages (to build packages from source code - particularly true for kernel modules)
- Package types
 - Package
 - RPM
 - Red Hat
 - CentOS
 - Fedora
 - Oracle Linux
 - SUSE Enterprise Linux
 - OpenSUSE
 - DEB

- Debian
 - Ubuntu
 - Mint
 - Knoppix
- Commands
 - `lsb_release -a` to find version of Operating System
 - When searching for packages for this version of the OS you can search by OS code name eg: `focal`
- Architectures
 - amd64 | x86_64
 - i386 | x86
 - arm (RISC5 Sakthi)
 - ppc64el | OpenPOWER
 - all | noarch | src (not tied to any architecture)
- Commands
 - `uname -a` gives the kernel version and the type of architecture.
- Tools
 - Package Type
 - RPM
 - Yellowdog Updater Modifier (yum)
 - Red Hat Package Manager (rpm)
 - Dandified YUM (dnf)
 - DEB
 - synaptic (GUI)
 - aptitude (Command Line)
 - Advanced Package Tool (apt)
 - dpkg
 - dpkg-deb
- Package management in Ubuntu using `apt`
 - Inquiring package db
 - Search packages for a keyword
 - `apt-cache search keyword`
 - List all packages
 - `apt-cache pkgnames`
 - `apt-cache pkgnames | sort | less` for page by page sorted display
 - `apt-cache pkgnames nm` for all packages starting with nm
 - Display package records of a package
 - `apt-cache show -a package`
- Package Names
 - Package
 - RPM
 - `package-version-release.architecture.rpm`
 - DEB
 - `package_version-revision_architecture.deb`
 - eg: `pool/universe/n/nmap/nmap_7.80+dfsg1-2build1_amd64.deb`
- Package Priorities
 - required : essential to proper functioning of the system
 - important : provides functionality that enables the system to run well
 - standard : included in a standard system installation
 - optional : can omit if you do not have enough storage
 - extra : could conflict with packages with higher priority, has specialized requirements, install only if needed.
 - Priority is displayed as `extra` in the output of `apt-cache show nmap` or `apt-cache show wget` for example.
- Package Sections
 - [Package Sections for Ubuntu focal](#)
 - `apt-cache show fortunes` shows Section : `universe/games`
- Checksums
 - For a small change in the original file the checksum is very different. This is useful to check if the original file has been tampered or not.
 - Can be used to verify that nothing has gone wrong to the contents of the file while downloading.
 - md5sum
 - 128 bit string
 - `md5sum filename`
 - SHA1
 - 160 bit string
 - `sha1sum filename`
 - SHA256
 - 256 bit string
 - `sha256sum filename`

4.2 * Who can install packages in Linux OS ? - administrators - sudoers in the case of Ubuntu - Only sudoers can install/upgrade/remove packages - a sudo command can be executed by those who are listed in `/etc/sudoers` - Command `sudo cat /etc/sudoers` . If the current `$USER` is not in the sudoers file the incident will be reported. - In the file the users listed under `# User privilege specification` have sudo permission. - sudo attempts and authentication failures get recorded in `/var/log/auth.log` . View using `sudo tail -n 100 /var/log/auth.log` * When installing a package the system knows the website/server from which the packages have to be downloaded - This information is stored in the folder `/etc/apt` - Uncommented lines in the file `sources.list` have the debian/ubuntu sources - A directory `sources.list.d` stores sources for third party software. Allows `apt update` to know new versions to download from repositories stored in these files - Synchronize package overview files - `sudo apt-get update` fetches updates and keeps them in cache - Upgrade all installed packages - `sudo apt-get upgrade` upgrades the packages. It lists how many updates are going to be affected and how much data is going to be downloaded. - `sudo apt autoremove` to remove unused packages that were earlier installed to satisfy a particular dependency but are not needed now. - Install a package - `sudo apt-get install packagename` - `sudo apt-get remove packagename` to remove a particular package - `sudo apt-get reinstall packagename` to fix problems caused by accidental file deletions. - Clean local repository of retrieved package files - `apt-get clean` - Purge package files from the system - `apt-get purge package` * Package

management in Ubuntu using `dpkg` - Allows installation directly from a `.deb` file. Package management at a lower level. - `/var/lib/dpkg` has some information about the packages - Files - `arch, available, status` - `cat arch` displays the architectures for which packages have been installed on the system - `amd64, i386` - `less available` displays list of packages with info. - `less status` displays if a particular package is installed or not - Folder - `info` - contains a set of files for each of the packages that have been installed - `ls wget*` will give files with information about `wget` - `more wget.conf` gives location of configuration file - `more wget.list` displays list of files that would get installed on the system with the package - `more wget.md4sums` displays the list of md5sums of the installed files. (Used to catch tampering) * Using `dpkg` - List all packages whose names match the pattern - `dpkg -l pattern` - List installed files that came from packages - `dpkg -L package` - Display/Report the status of packages - `dpkg -s package` - Search installed packages for a file - `dpkg -S pattern` - eg: `dpkg -S /usr/bin/perl` shows the package from which the executable has come. ie: `perl-base` - To query the `dpkg` database about all the packages - `dpkg-query` - Example `dpkg-query -W -f='${Section} ${binary:Package}\n' | sort | less` - Example where output is filtered `dpkg-query -W -f='${Section} ${binary:Package}\n' | grep shells` * Installing a deb package - `dpkg -i package_version-revision_architecture.deb` - not a good idea since it may have some dependencies that will have to be taken care of manually - Do not download deb files from unknown sources and install it on the system - By default use package management pointing to a reliable repository - Uninstalling packages using `dpkg` is NOT recommended. You may be removing a package that is required by many other packages. * When compatibility issues cannot be resolved one can use `snap` or `docker` as alternatives when you are unable to install a particular version of a package.

4.3

Pattern Matching

- Regular Expressions `regex` and `grep` commands
 - POSIX standard
 - IEEE 1003.1-2001 IEEE Standard for IEEE Information Technology – Portable Operating System Interface (POSIX(TM))
 - [Refer](#)
 - POSIX defines regular expressions to be of 2 different types - Basic and Extended.
- Regex
 - `regex` is a pattern template to filter text
 - BRE: POSIX Basic Regular Expression engine
 - ERE: POSIX Extended Regular Expression engine
- Why learn regex?
 - Process some input from the user or perform some string operations.
 - Languages: Java, Perl, Python, Ruby, ...
 - Tools: `grep`, `sed`, `awk`, ...
 - Applications: MySQL, PostgreSQL, ...
- Usage
 - `grep 'pattern' filename` - to operate on every line in the file
 - `command | grep 'pattern'`
 - the `grep` command operates line after line. A common feature in many utilities in linux.
 - enclose pattern in single quotes
 - Default engine: BRE
 - Switch to use ERE in 2 ways:
 - `egrep 'pattern' filename`
 - `grep -E 'pattern' filename`
- ### Special characters (BRE & ERE)

Character	Description
<code>.</code>	Any single character except null or newline
<code>*</code>	Zero or more of the preceding character / expression
<code>[]</code>	Any of the enclosed characters; hyphen (-) indicates character range
<code>^</code>	Anchor for beginning of line or negation of enclosed characters
<code>\$</code>	Anchor for end of line
<code>\</code>	Escape special characters

- ### Special characters (BRE)

Character	Description
<code>\{n,m\}</code>	Range of occurrences of preceding pattern at least n and utmost m times
<code>\(\)</code>	Grouping of regular expressions

- ### Special characters (ERE)

Character	Description
-----------	-------------

Character	Description
<code>{n,m}</code>	Range of occurrences of preceding pattern at least n and utmost m times
<code>()</code>	Grouping of regular expressions
<code>+</code>	One or more of preceding character / expression
<code>?</code>	Zero or one of preceding character / expression
<code>\ </code>	Logical OR over the patterns

- **### Character Classes**

Class	Description
<code>[:print:]</code>	Printable
<code>[:alnum:]</code>	Alphanumeric
<code>[:alpha:]</code>	Alphabetic
<code>[:lower:]</code>	Lower case
<code>[:upper:]</code>	Upper case
<code>[:digit:]</code>	Decimal digits
<code>[:blank:]</code>	Space / Tab
<code>[:space:]</code>	Whitespace
<code>[:punct:]</code>	Punctuation
<code>[:xdigit:]</code>	Hexadecimal
<code>[:graph:]</code>	Non-space
<code>[:cntrl:]</code>	Control characters

- **Backreferences**
 - `\1` through `\9`
 - `\n` matches whatever was matched by nth earlier parenthesized subexpression
 - A line with two occurrences of hello will be matched using: `\(hello\).*\1`

- **BRE operator precedence**

Highest to Lowest
<code>[..] [=] [::]</code> char collation
<code>\metachar</code>
<code>[]</code> Bracket expansion
<code>()</code> \n subexpresions and backreferences
<code>* {}</code> Repetition of preceding single char regex
Concatenation
<code>^ \$</code> anchors

- ### ERE operator precedence

Highest to Lowest
[.] [=] [::] char collation
\metachar
[] Bracket expansion
() grouping
* + ? { } Repetition of preceding regex
Concatenation
^ \$ anchors
alternation

- ### Examples using grep
 - [Example File names.txt \(Containing Names/Roll-No\)](#)
 - Basic use
 - `grep 'Raman' names.txt` matches line with Raman Singh
 - `cat names.txt | grep 'ai'` matches line with Snail
 - Usage of `.`
 - `cat names.txt | grep 'S.n'` matches lines with Singh and Sankaran
 - Usage of `$`
 - `cat names.txt | grep '.am$'` matches lines that end with xam
 - Escaping a `.`
 - `cat names.txt | grep '\.'` matches lines that have a `.`
 - Using anchors at the beginning
 - `cat names.txt | grep '^M'` matches lines beginning with m
 - Case insensitive matching with the `i` flag
 - `cat names.txt | grep -i '^e'` matches lines beginning with e or E.
 - Word boundaries `\b`
 - `cat names.txt | grep 'am\b'` matches lines with words that end with 'am'
 - Use of square brackets `[]` to give options
 - `cat names.txt | grep 'M[ME]'` matches lines containing 'MM' or 'ME'
 - `cat names.txt | grep '\bS.*[mn]'` matches lines containing words beginning with S and ending with m or n.
 - `cat names.txt | grep '[aeiou][aeiou]'` matches lines that have 2 vowels side by side
 - `cat names.txt | grep 'B90[1-4]'` matches words beginning with B90 and ending with range 1-4.
 - `cat names.txt | grep 'B90[^1-4]'` matches words beginning with B90 and ending with characters other than the range 1-4. A hat inside square brackets implies negation
 - Specifying occurrences using escaped braces
 - `cat names.txt | grep 'M{2\}'` matches lines which have 'MM'
 - `cat names.txt | grep 'M{1,2\}'` matches lines which have one or 2 'M's
 - Grouping patterns that are matched using parenthesis. Repeating whatever is matched by using `\1`
 - `cat names.txt | grep '\(ma\)'` matches lines containing 'ma'
 - `cat names.txt | grep '\(ma\).*\1'` matches a pattern beginning with 'ma' and ending with 'ma' eg: U'mair Ahma'd. The `\1` back-references the first parenthesis.
 - `cat names.txt | grep '\(.a\).*\1'` matches a pattern like 'Mary Ma'nickam
 - `cat names.txt | grep '\(a.\\)\{3\}'` matches a pattern like 'S'agayam'
 - Using Extended Regular Expression Engine
 - `cat names.txt | egrep 'M+'` will match lines where M occurs one or more times.
 - `cat names.txt | egrep '^M+'` will match lines where M occurs one or more times at the beginning of a line.
 - `cat names.txt | egrep '^M*'`
 - `cat names.txt | egrep '^M*a'` matches lines where 'M' may or may not occur followed by 'a'
 - `cat names.txt | egrep '^M.*a'` matches lines where 'M' has to occur at the beginning of a line followed by any number of characters and ending with 'a'
 - Watch out for the interpretation of `*`
 - `cat names.txt | egrep '(ma)+'` 'ma' could occur one or more times.
 - `cat names.txt | egrep '(ma)*'` 'ma' could occur zero or more times.
 - Use of pipe as an alternation between 2 patterns of strings to be matched
 - `cat names.txt | egrep '(ED|ME)'` matches lines containing 'ED' or 'ME'
 - `cat names.txt | egrep '(Anu|Raman)'` matches lines containing 'Anu' or 'Raman'. Length of string on both sides of pipe need not be the same.
 - `cat names.txt | egrep '(am|an)$'` matches lines containing 'am' or 'an' at the end.

4.4 * ### More Examples using grep and egrep - Get package names that are exactly 4 characters long - `dpkg-query -W -f'${Section} ${binary:Package}\n' | egrep '.{4}$'` - Get package names that are from the math section - `dpkg-query -W -f'${Section} ${binary:Package}\n' | egrep '^math' -`
[Example File chartype.txt \(Containing few lines with control character\)](#) - control character inserted using `echo '$\cc' >> chartype.txt` - get lines that have an

alphanumeric character at the beginning of the line - `cat chartype.txt | grep '^[:alnum:]'` - get lines that have digits at the end of the line - `cat chartype.txt | grep '[:digit:]]$'` - get lines that have a ctrl character - `cat chartype.txt | grep '[:ctrl:]'` - `cat chartype.txt | grep -v '[:cntrl:]'` will show the reverse including the empty lines - get lines that do not have a ctrl character - `cat chartype.txt | grep '[^[:cntrl:]']` (This does not work as intended) - get lines that have printable characters (exclude blank lines) - `cat chartype.txt | grep '[:print:]'` - get lines that have blank space characters (exclude blank lines) - `cat chartype.txt | grep '[:blank:]'` - `[:graph:]` is used to match any non space character - To skip blank lines - `cat chartypes.txt | egrep -v '^$'` Here -v excludes and '^\$' captures empty lines - Identify a line with a 12 digit number - `egrep '[:digit:]]{12}' patterns.txt` - Identify a line with a 6 digit number (Use word boundaries) - `egrep '\b[:digit:]]{6}\b' patterns.txt` - Match lines containing Roll Number of the form MM22B001 - `egrep '\b[:alpha:]]{2}[:digit:]]{2}[:alpha:]][:digit:]]{3}\b' patterns.txt` - Match urls without the http - `egrep '\b[:alnum:]]+\.[:alnum:]]+\b' patterns.txt` - **Trimming text** - top to bottom using `head` and `tail` - sideways or horizontal trimming of lines using `cut` - `cut -c 1-4 fields.txt` displays only first 4 characters. Can also use -4 for beginning to 4th place or -2 to cut from 2nd place to end. - `cat fields.txt | cut -d " " -f 1` - This uses " " as a delimiter -d and prints only the first field - `cat fields.txt | cut -d ' ' -f 1-2` - to get both fields - Capture hello world from 1234;hello world, line 1 - `cat fields.txt | cut -d ';' -f 2 | cut -d "," -f 1` - `egrep ';\.*, ' fields.txt` (To trim pass the output of `grep` to `sed`) - Combining this with top to bottom trimming - `cat fields.txt | cut -d ';' -f 2 | cut -d "," -f 1 | head -n 2 | tail -n 1`

• Own experiments using regex

- Get strictly alphanumeric words
 - `cat test.txt | egrep '\b([a-z]+[0-9]+|[0-9]+[a-z]+)\b'`

• REPLIT Code with Us session

- Getting files with a specific permission pattern from a file
 - `cat lsinfo.txt | grep 'rw-r--r--'` ;
- Get all files excluding directories in lsinfo.txt whose last modified date is in January
 - `cat lsinfo.txt | grep '^[\d].*Jan'`
- To count the number of lines that starts with a capital letter and contains the word it (case-sensitive)
 - `cat twocities.txt | grep -c '^[:upper:]].*\bit\b'`
- to display all the lines that does not contain the word "we" in it
 - `cat twocities.txt | egrep -v '\bwe\b'`
- using `cut` to display only the countries and its capitals of file.txt in the format Country, Capital (eg in file.txt : India, New Delhi; Asia)
 - `cat file.txt | cut -d ';' -f 1`
- all the countries in the file file.txt sorted alphabetically by name in reverse order
 - `cat file.txt | cut -d ' ' -f 1 | sort -r`
- `cut` command to extract the continents (including the one white space in the beginning) of the first 5 lines of file.txt and store it in another file named continent.txt
 - `head -n 5 file.txt | cut -d ';' -f 2 > continent.txt`
- list the names of all the c++ files in the current directory which contains a line such that the line starts with the string void main() and ends with the character {. There should be one or more spaces/tabs between the characters { and }.
 - `egrep '^void[:space:]]main\(\)[:space:]]+{$' *.cpp | cut -d '.' -f 1`
 - `grep '^void[:space:]]main()[:space:]][:space:]]*{$' *.cpp | cut -d '.' -f 1`
- print the count of these files in the following line
 - `egrep -l '^void[:space:]]main\(\)[:space:]]+{$' *.cpp | tee /dev/tty | wc -l`
 - `|tee /dev/tty` is used to print the output to terminal and also pipe the output to the next command.
 - -l flag for `grep` and `egrep` prints the name of each input file that matches
- command to list all the packages installed on your machine and their versions in the format Package Version in a sorted manner
 - `dpkg-query -W -f='${Package} ${Version}\n' | sort`

Week 5 Notes

Command Line Editors

- Working with text files in the terminal
- Editors
 - Line Editors (Present in almost every flavour of UNIX / GNU Linux)
 - `ed`
 - `ex` (improved version of `ed`)
 - Terminal Editors
 - `pico` (Came along with the pine email application)
 - `nano` (Features added to `pico`)
 - `vi` (most popular and complex)
 - `emacs`
 - GUI Editors
 - KDE
 - `kate`
 - `kwrite`
 - GNOME
 - `gedit`
 - `sublime`
 - `atom` (popular among github users)
 - `brackets` (Popular for those writing html code)
 - IDE
 - `eclipse`
 - `Bluefish`
 - `NetBeans`
- Features of text editors
 - Scrolling , view modes, current position in file
 - Navigation (char,word,line,pattern)
 - Insert, Replace, Delete
 - Cut-Copy-Paste

- Search-Replace
- Language-aware syntax highlighting
- Key-maps, init scripts, macros
- Plugins
- Both `vi` and `emacs` editors satisfy all the above requirements

ed commands

Action	Command
Show the Prompt	<code>P</code>
Command Format	<code>[addr[,addr]]cmd[params]</code>
Commands for location	<code>2 . \$ % + - , ; /RE/</code>
Commands for editing	<code>f p a c d i j s m u</code>
Execute a Shell command	<code>!command</code>
edit a file	<code>e filename</code>
read file contents into buffer	<code>r filename</code>
read command output into buffer	<code>r !command</code>
write buffer to filename	<code>w filename</code>
quit	<code>q</code>

Using ed

- `man ed` doesn't give much info. Use `info ed`
- `ed test.txt` shows a number indicating number of bytes read into memory
- `1` displays the first line
- `$` displays the last line
- `,p` and `%p` shows the contents of the entire buffer
- `2,3p` range - 2nd to 3rd line
- `/hello/` matches and shows first occurrence of the pattern
- `+` and `-` to scroll by line
- `;p` from current position to end of buffer
- `.` displays the current line
- `!date` running the date command within `ed`
- `r !date` read output of date command to buffer at current position
- `w` writes the file (saves it)
- `d` delete current line
- `a` to append after current line. Press `.` and `enter` when done
- `s/appended/Appended/` Substitute - Search and replace from current line.
- `f` shows the name of the file being edited
- `p` shows the contents of the current line
- `j` for joining lines. Usage `5,6j` to join line 5 and 6
- `m` to move a line to a particular position. Usage `m1` to move current line to just below line 1. `m0` to move it right to the top
- `u` to undo previous change
- To add something to every line `%s/(.*\)/PREFIX \1/`
 - `\1` is the back substitution
 - `\(.*\)` indicates any character that can be matched
 - `PREFIX` is the replacement string
- `3,5s/PREFIX/prefix/` substitutes prefix for PREFIX from line 3 to 5

Commands for editing in ed / ex

Command	Action
<code>f</code>	show name of file being edited
<code>p</code>	print the current line
<code>a</code>	append at the current line

Command	Action
c	change the line
d	delete the current line
i	insert line at the current position
j	join lines
s	search for regex pattern
m	move current line to position
u	undo latest change

Using nano

- [Link](#)

Using vi

- [Link](#)

Using emacs

* [Link](#)

Scripts

- Software Tools Principles (Ref: Classic Shell Scripting – Arnold Robbins & Nelson H.F. Beebe)
 - Do one thing well
 - Process lines of text, not binary
 - Use regular expressions
 - Default to standard I/O
 - Don't be chatty
 - Generate same output format accepted as input
 - Let someone else do the hard part
 - Detour to build specialized tools

```

#! interpreter
# comments
commands
loops
variables
case statements
functions

```

- program - shell , awk , sed , python , ruby , perl
- script
 - sourced
 - `. scriptname` `source scriptname`
 - PID same as the current shell commands are executed one after other shell environment continues
 - Used to prepare environment
 - executed
 - `./scriptname`
 - Needs execution permission
 - New process gets created to run script
 - PID is not same as the shell commands are executed one after other
 - New environment lost after return
 - Used to create a new functionality
- Script location
 - Use absolute path or relative path while executing the script
 - Keep the script in folder listed in \$PATH
 - Watch out for the sequence of directories in \$PATH
- bash environment
 - Login shell
 - `/etc/profile` `~/.bash_profile` `~/.bash_login` `~/.profile`
 - Non-login shell
 - `/etc/bash.bashrc` `~/.bashrc`
- Output from shell scripts
 - echo

- simple
- terminates with a newline if -n option not given
- echo My home is \$HOME
- printf
- supports format specifiers like in C
- printf "My home is %s\n" \$HOME
- Input to shell scripts
 - read var
 - string read from command line is stored in \$var
- Shell Script arguments
 - \$0 name of the shell program
 - \$# number of arguments passed
 - \$1 or \${1} first argument
 - \${11} eleventh argument
 - \$* or @\$ all arguments at once
 - "\$*" all argument as a single string
 - "\$@" all argument as a separate strings
 - example: ./myscript.sh -l arg2 -v arg4
- Command substitution
 - var=`command`
 - var=\$(command)
 - command is executed and the output is substituted.
 - Here, the variable var will be assigned with that output.
- for do loop
 - for var in list do commands done
 - commands are executed once for each item in the list
 - space is the field delimiters
 - set IFS if required. If the field separator is different from space.
- case statement
 - case var in pattern1) commands ;; pattern2) commands ;; esac
 - commands are executed each pattern matched for var in the options
- if loop
 - if condition then commands fi
 - if condition; then commands fi
 - commands are executed only if condition returns true
- Conditions
 - test expression
 - test -e file
 - [exprn]
 - [-e file]
 - [[exprn]]
 - [[\$ver == 5.*]]
 - ((exprn))
 - ((\$v ** 2 > 10))
 - command
 - wc -l file
 - pipeline
 - who|grep "joy" > /dev/null
 - For negation ! condition
- String Comparison , Numeric, file comparison
 - expressions
 - unary
 - binary
- test numeric comparisons
 - Table | Comparison | Description | |--| \$n1 -eq \$n2 | Check if n1 is equal to n2 || \$n1 -ge \$n2 | Check if n1 is greater than or equal to n2 || \$n1 -gt \$n2 | Check if n1 is greater than n2 || \$n1 -le \$n2 | Check if n1 is less than or equal to n2 || \$n1 -lt \$n2 | Check if n1 is less than n2 || \$n1 -ne \$n2 | Check if n1 is not equal to n2 |
- test string comparisons
 - Table | Comparison | Description | |--| \$str1 = \$str2 | Check if str1 is same as str2 || \$str1 != \$str2 | Check if str1 is not same as str2 || \$str1 < \$str2 | Check if str1 is less than str2 || \$str1 > \$str2 | Check if str1 is greater than str2 || -n \$str2 | Check if str1 has length greater than zero || -z \$str2 | Check if str1 has length of zero |
- Unary file comparisons
 - Table | Comparison | Description | |--| -e file | Check if file exists || -d file | Check if file exists and is a directory || -f file | Check if file exists and is a file || -r file | Check if file exists and is readable || -s file | Check if file exists and is not empty || -w file | Check if file exists and is writable || -x file | Check if file exists and is executable || -O file | Check if file exists and is owned by current user || -G file | Check if file exists and default group is same as that of current user |
- Binary file comparisons
 - Table | Comparison | Description | |--| file1 -nt file2 | Check if file1 is newer than file2 || file1 -ot file2 | Check if file1 is older than file2 |
- while do loop
 - while condition do commands done
 - commands are executed only if condition returns true
- until do loop
 - until condition do commands done
 - commands are executed only if condition returns false
- functions
 - definition
 - myfunc() { commands }

- call
- myfunc
- commands are executed eachtime myfunc is called
- Definitions must be before the calls
- Demo
 - bash #! /bin/bash # s1.sh is my first script echo I am invoked as echo \$0 echo hello world echo the PID of the process running this script is : echo \$\$ ps --forest export myvar=MYVAR echo \$myvar
 - Source it using . s1.sh or source s1.sh . This displays the same PID as the Bash terminal
 - Executing this using ./s1.sh displays an error as there is no executable permission.
 - Provide executable permission using chmod 755 s1.sh and then run using ./s1.sh
 - Ths time the PID is different.
 - ps --forest shows all the processes that are running and the spawned processes.
 - A variable set during execution in a subshell will not be available in the parent shell. If the script is sourced the variable will be available.
 - \$0 displays which ever way the script has been invoked (absolute or relative path or just the name of the script)
 - ./s1.sh -l arg2 will show -l as \$1 and arg2 as \$2
 - bash #! /bin/bash # s1.sh modified script echo Number of arguments echo \$# echo First argument echo \$1 echo Second argument echo \$2 if test \$1 = \$2; then echo The arguments are the same fi
 - Executing the above script using ./s1.sh hello hello will say that the arguments are the same.
 - bash #! /bin/bash echo use of for loop for i in arg1 arg2 arg3 do echo \$i done
 - The above script just prints arg1,arg2 and arg3 on 3 lines
 - bash #! /bin/bash echo use of for loop for i in file_{1..9} do echo \$i done
 - prints file_1,file_2 ... file_9
 - bash #! /bin/bash echo use of for loop for i in file_{A..D}{1..9} do echo \$i done
 - prints 36 lines
 - bash #! /bin/bash echo use of for loop for i in \$(ls /bin/z*) do echo \$i done
 - shows each file in bin directory starting with z
 - file znew | grep "shell script" identifies whether the file passed (in bin) is a shell script
 - bash #! /bin/bash echo Shell Scripts in bin directory for i in \$(ls /bin) do #echo /bin/\$i file /bin/\$i | grep "shell script" done
 - Prints the files which are shell scripts in the bin directory

6.1

Shell programming

More features in bash scripts

- Debugging
 - Print the command before executing it
 - set -x ./myscript.sh
 - bash -x ./myscript.sh
 - Place the set -x inside the script
- Combining conditions
 - [\$a -gt 3] && [\$a -gt 7]
 - [\$a -lt 3] || [\$a -gt 7]
 - Example [condition-examples.sh](#)
- Shell arithmetic
 - Using let
 - let a=\$1+5
 - let "a= \$1 + 5"
 - Using expr
 - expr \$a +20
 - expr "\$a + 20"
 - b=\$(expr \$a + 20)
 - Using \$[expression]
 - b=\$[\$a + 10]
 - Using \$((expression))
 - b=\$((\$a + 10))
 - ((b++)) - Without \$. Not intending to return. Useful for incrementing
 - Example [arithmetic-example-1.sh](#)
- ### expr command operators

Expression	Description
a + b	Return arithmetic sum of a and b
a - b	Return arithmetic difference of a and b
a * b	Return arithmetic product of a and b
a / b	Return arithmetic quotient of a divided by b

Expression	Description
<code>a % b</code>	Return arithmetic remainder of a divided by b
<code>a > b</code>	Return 1 if a greater than b; else return 0
<code>a >= b</code>	Return 1 if a greater than or equal to b; else return 0
<code>a < b</code>	Return 1 if a less than b; else return 0
<code>a <= b</code>	Return 1 if a less than or equal to b; else return 0
<code>a = b</code>	Return 1 if a equals b; else return 0
<code>a \ b</code>	Return a if neither argument is null or 0; else return b
<code>a & b</code>	Return a if neither argument is null or 0; else return 0
<code>a != b</code>	Return 1 if a is not equal to b; else return 0
<code>str : reg</code>	Return the position upto anchored pattern match with BRE str
<code>match str reg</code>	Return the pattern match if reg matches pattern in str
<code>substr str n m</code>	Return the substring m chars in length starting at position n
<code>index str chars</code>	Return position in str where any one of chars is found else return 0
<code>length str</code>	Return numeric length of string str
<code>+ token</code>	Interpret token as string even if its a keyword
<code>(exprn)</code>	Return the value of expression exprn

- Example [expr-examples.sh](#)
- Bench Calculator
 - An arbitaty preciscion calculator language
 - `bc -l`
 - the `-l` option loads the math library.
 - `12^6` or `12.6/3.6`
 - Can be used for floating point operations.
- heredoc feature
 - helps while passing long strings without having to worry about `\n` etc.
 - Example [heredoc-example-1.sh](#)
 - Example [heredoc-example-2.sh](#)
 - A hyphen tells bash to ignore leading tabs
- PATH variable
 - Example [path-example.sh](#)
 - IFS (Internal Field Separator)

L6.3

- if-elif-else-fi loop

```

if condition1
then
    commandset1
else
    commandset2
fi

```

```

if condition1
then
    commandset1
elif condition2
then
    commandset2
elif condition3
then
    commandset3
else
    commandset4
fi

```

- case statement options
 - commandset4 is the default for values of \$var not matching what are listed

```

case $var in
    op1)
        commandset1;;
    op2 | op3)
        commandset2;;
    op4 | op5 | op6)
        commandset3;;
    *)
        commandset4;;
esac

```

- c style for loop : one variable
 - extension of POSIX and maynot be available in all the shells
 - Adding `time` before a script command gives the amount of time taken to execute.

```

begin=1
finish=10
for (( a = $begin; a < $finish; a++ ))
do
    echo $a
done

```

- c style for loop : two variables
 - Note: Only one condition to close the for loop

```

begin1=1
begin2=10
finish=10
for (( a=$begin1, b=$begin2; a < $finish; a++, b-- ))
do
    echo $a $b
done

```

- processing output of a loop
 - Output of the loop is redirected to the tmp file

```

filename=tmp.$$
begin=1
finish=10
for (( a = $begin; a < $finish; a++ ))
do
    echo $a
done > $filename

```

- break
 - Break out of inner loop

```

n=10
i=0
while [ $i -lt $n ]
do
    echo $i
    (( i++ ))
    if [ $i -eq 5 ]
    then
        break
    fi
done

```

- Break out of outer loop

```

n=10
i=0
while [ $i -lt $n ]
do
    echo $i
    j=0
    while [ $j -le $i ]
    do
        printf "$j "
        (( j++ ))
        if [ $j -eq 7 ]
        then
            break 2
        fi
    done
    (( i++ ))
done

```

- continue
 - Continue will skip rest of the commands in the loop and goes to next iteration

```

n=9
i=0
while [ $i -lt $n ]
do
    printf "\n loop $i:"
    j=0
    (( i++ ))
    while [ $j -le $i ]
    do
        (( j++ ))
        if [ $j -gt 3 ] && [ $j -lt 6 ]
        then
            continue
        fi
        printf "$j "
    done
done

```

- shift
 - shift will shift the command line arguments by one to the left.
 - shift is destructive - after the arguments are shifted to the left they are gone. This is only helpful if you don't need the arguments later.
 - n checks if it is a non-zero argument
 - except \$0, which is the name of the script, the rest of the arguments get popped

```
i=1
while [ -n "$1" ]
do
    echo argument $i is $1
    shift
    (( i++ ))
done
```

- **exec**
 - `exec ./my-executable --my-options --my-args`
 - To replace shell with a new program or to change i/o settings
 - If new program is launched successfully, it will not return control to the shell
 - If new program fails to launch, the shell continues
- **eval**
 - `eval my-arg`
 - Execute argument as a shell command
 - Combines arguments into a single string
 - Returns control to the shell with exit status
 - Example [eval-example.sh](#)
- **function**
 - Example [function-example.sh](#)
- **getopts**
 - This script can be invoked with only three options: `a` , `b` , `c` . The options `b` and `c` will take arguments.
 - Example [getopts-example.sh](#)

```
while getopts "ab:c:" options;
do
    case "${options}" in
        b)
            barg=${OPTARG}
            echo accepted: -b $barg
            ;;
        c)
            carg=${OPTARG}
            echo accepted: -c $carg
            ;;
        a)
            echo accepted: -a
            ;;
        *)
            echo Usage: -a -b barg -c carg
            ;;
    esac
done
```

- **select loop**
 - Text Menu
 - Example [select-example.sh](#)

```
echo select a middle one
select i in {1..10}
do
    case $i in
        1 | 2 | 3)
            echo you picked a small one;;
        8 | 9 | 10)
            echo you picked a big one;;
        4 | 5 | 6 | 7)
            echo you picked the right one
            break;;
    esac
done
echo selection completed with $i
```

- **Additional notes**
 - Warning : Never `eval` a user supplied string on any command line
 - `eval` is sending the strings to the shell and printing them out.

- Can source a file with functions to use it in a shell script
- `source mylib.sh`
- Do not give set uid permission to the scripts unless you know what you are doing

L6.4

awk

A language for processing fields and records

- Introduction
 - awk is a programming language, quick to code and fast in execution
 - awk is an abbreviation of the names of three people who developed it: **Aho, Weinberger & Kernighan**
 - It is a part of POSIX, IEEE 1003.1-2008
 - Variants: nawk, gawk, mawk ...
 - gawk contains features that extend POSIX (normally seen on GNU Linux systems with a symbolic link from awk to gawk)
 - Though awk is viewed as a scripting language it has enough mathematical functions to use for routine calculations. IT can do things that spreadsheets cannot do.
- Execution model
 - Input stream is a set of records
 - Eg., using "\n" as record separator, lines are records
 - Each record is a sequence of fields
 - Eg., using " " as field separator, words are fields. Even a regular expression can be used as an FS.
 - Splitting of records to fields is done automatically
 - Each code block executes on one record at a time, as matched by the pattern of that block
- Usage
 - Single line at the command line
 - `cat /etc/passwd | awk -F":" '{print $1}'`
 - Script interpreted by awk
 - `./myscript.awk /etc/passwd`
 - `myscript.awk awk #!/usr/bin/gawk -f BEGIN { FS=":" } { print $1 }`
- Examples
 - [block-ex-1.awk](#)
 - `./block-ex-1.awk block-ex-1.input`
 - `cat block-ex-1.input | ./block-ex-1.awk`
 - For each line Default block will be processed once.
 - You can have as many begin and end blocks wherever required in the awk script. BEGIN will be processed before the default block and and END will be processed after the default block.
 - We don't need ; at the end of every statement unless you need to write multiple statements on a single line.
 - `$0` represents the line(record) which is currently being processed.
- Built-in variables

Variable	Description
ARGC	Number of arguments supplied on the command line (except those that came with -f & -v options)
ARGV	Array of command line arguments supplied; indexed from 0 to ARGC-1
ENVIRON	Associative array of environment variables
FILENAME	Current filename being processed
FNR	Number of the current record, relative to the current file
FS	Field separator, can use regex
NF	Number of fields in the current record
NR	Number of the current record
OFMT	Output format for numbers
OFS	Output fields separator
ORS	Output record separator

Variable	Description
RS	Record separator
RLENGTH	Length of string matched by match() function
RSTART	First position in the string matched by match() function
SUBSEP	Separator character for array subscripts
\$0	Entire input record
\$n	nth field in the current record

- **awk scripts**

- ##### pattern {procedure}
- pattern (optional. If not given the code block is called default block and it is applied to every line in the input stream.)
 - **BEGIN**
 - **END**
 - general expression
 - **regex**
 - Relational Expression
 - Pattern-matching expression
- procedure (will be applied to all recors that match the pattern)
 - Variable assignment
 - Array assignment
 - Input / output commands
 - Build-in functions
 - User-defined functions
 - Control loops

- **Execution**

- BEGIN { commands; }
 - Executed once, before files are read
 - Can appear anywhere in the script
 - Can appear multiple times
 - Can contain program code
- END { commands; }
 - Executed once, after files are read
 - Can appear anywhere in the script
 - Can appear multiple times
 - Can contain program code
- pattern { commands; }
 - Patterns can be combined with && || !
 - Range of records can be specified using comma
 - Executed each record pattern evalutes to true
 - Script can have multiple such blocks
- { commands; }
 - Executed for all records
 - Can have multiple such blocks

- **operators**

- Assignment
 - = += -= *= /= %= ^= **=
- Logical
 - || &&
- Algebraic
 - + - * / % ^ **
- Relational
 - > <= > >= != ==

Operation	Description
expr ? a : b	Conditional expression
a in array	Array membership
a ~ /regex/	Regular expression match
a !~ /regex/	Negation of regular expression match

Operation	Description
<code>++</code>	Increment, both prefix and postfix
<code>--</code>	decrement, both prefix and postfix
<code>\$</code>	Field reference
<code></code>	Blank is for concatenation

- Adding 0 to a string makes it get interpreted as a number.

- Functions and commands

Operation	Commands
Arithmetic	<code>atan2</code> <code>cos</code> <code>exp</code> <code>int</code> <code>log</code> <code>rand</code> <code>sin</code> <code>sqrt</code> <code>srand</code>
String	<code>asort</code> <code>asorti</code> <code>gsub</code> <code>index</code> <code>length</code> <code>match</code> <code>split</code> <code>sprintf</code> <code>strtonum</code> <code>sub</code> <code>substr</code> <code>tolower</code> <code>toupper</code>
Control Flow	<code>break</code> <code>continue</code> <code>do</code> <code>while</code> <code>exit</code> <code>for</code> <code>if</code> <code>else</code> <code>return</code>
Input / Output	<code>close</code> <code>fflush</code> <code>getline</code> <code>next</code> <code>nextline</code> <code>print</code> <code>printf</code>
Programming	<code>extension</code> <code>delete</code> <code>function</code> <code>system</code>
bit-wise	<code>and</code> <code>compl</code> <code>lshift</code> <code>or</code> <code>rshift</code> <code>xor</code>

- Example
 - [block-ex-2.awk](#)
 - [block-ex-3.awk](#)
 - Blocks get executed based on whether the line has `alpha`, `alnum` or `digits`
 - [block-ex-4.awk](#)
 - Matching only the first field in the record with a pattern
 - [block-ex-5.awk](#)
 - Field Separator as regular expression
 - Number of fields as condition

L6.4

- arrays
 - Associative arrays
 - Sparse storage
 - Index need not be integer
 - `arr[index]=value`
 - `for (var in arr)`
 - `delete arr[index]`

- Loops

```
for (a in array)
{
    print a
}
```

```
if (a > b)
{
    print a
}
```

```
for (i=1;i<n;i++)
{
    print i
}
```

```
while (a < n)
{
    print a
}
```

```
do
{
    print a
} while (a<n)
```

- Example
 - [block-ex-6.awk](#)
- ##### Functions
 - `cat infile | awk -f mylib -f myscrip.awk`
 - `mylib function myfunc1() { printf "%s\n", $1 } function myfunc2(a) { return a*rand() }`
 - `myscrip.awk BEGIN { a=1 } { myfunc1() b = myfunc2(a) print b }`
- Example
 - [func-example.awk](#)
 - [func-lib.awk](#)
- Pretty printing
 - `printf "format", a, b, c`
 - `format - %[modifier]control-letter`
 - `modifier`
 - `width`
 - `prec`
 - `-`
 - `control-letter`
 - `c` ascii char
 - `d` integer
 - `i` integer
 - `e` scientific notation
 - `f` floating notation
 - `g` shorter of scientific & float
 - `o` octal value
 - `s` string text
 - `x` hexadecimal value
 - `X` hexadecimal value in caps
- **bash + awk**
 - Including awk inside shell script
 - heredoc feature
 - Use with other shell scripts on command line using pipe
- Examples
 - Spreadsheet handling efficiency demo
 - [rsheet-create.awk](#)
 - `echo " " | ./rsheet-create.awk`
 - Time the process for 2 million records `time ./rsheet-create.awk emptyfile > rsheet-data.txt`
 - [rsheet-process.awk](#)
 - `time ./rsheet-process.awk rsheet-data.txt > rsheet-pdata.txt`
 - Analysing the server logs of <https://semantic.iitm.ac.in/>
 - [access-full.log](#) (80MB)
 - Making smaller files
 - `head access-full.log > access-head.log` and `tail access-full.log > access-tail.log`
 - To print ip addresses alone `awk BEGIN{FS=" "}{print $1} access-head.log`
 - To print only the date and ip `awk BEGIN{FS=" "}{d=substr($4,2,11);print d, $1} access-head.log`
 - The date from 5 days ago - `date --date="5 days ago" +%d/%m/%Y`
 - Use the above date string to extract details from the Apache log book
 - [apache-log-example-1.awk](#)
 - `./apache-log-example-1.awk access-head.log`
 - Investigate a suspicious ip address using `dig`
 - `dig -x 136.123.209.54`
 - `dig +noall +answer -x 34.234.167.93` for a one line output
 - [apache-log-example-2.awk](#)
 - `time ./apache-log-example-2.awk access-full.log > nstats.txt`

sed

- Introduction
 - It is a programming language for processing text streams
 - sed is an abbreviation for **s**tream **e**ditor
 - It is a part of POSIX

- sed precedes awk
- use sed to pre-process input for further processing
- sed is a meant for text processing, fast in execution
- sed is available everywhere !

- Execution model

- Input stream is a set of lines
- Each line is a sequence of characters
- Two data buffers are maintained: active **pattern** space and auxiliary **hold** space
- For each line of input, an **execution cycle** is performed loading the line into the pattern space
- During each cycle, all the statements in the script are executed in the sequence for matching **address pattern** for **actions** specified with the **options** provided

- usage

- Single line at the command line
- `sed -e 's/hello/world/g' input.txt`
- Script interpreted by sed
- `sed -f ./myscript.sed input.txt`
- `myscript.sed`

```
#!/usr/bin/sed -f
2,8s/hello/world/g
```

- sed statements

- `:label address pattern action options ;`
 - `address pattern`
 - `address`
 - `address, range`
 - `negation !`
 - `action`
 - Single Character action. Same as "ed" or "ex"
 - `options`
 - Depends on the `action`

- Grouping commands

- `{ cmd; cmd; }`

- address

- Selecting by Numbers
 - `5`
 - `$`
 - `%`
 - `1~3`
- Selecting by matching
 - `/regexp/`
- Range Address
 - `/regexp1/,/regexp2/`
 - `/regexp/, +4`
 - `/regexp/, ~2`
 - `5,15`
 - `5,/regexp/`

- actions

command	Description
<code>p</code>	Print the pattern space
<code>d</code>	Delete the pattern space
<code>s</code>	Substitute using regex match <code>s/pattern/replacement/g</code>
<code>=</code>	Print current input line number, \n
<code>#</code>	comment
<code>i</code>	Insert above current line
<code>a</code>	Append below current line
<code>c</code>	Change current line

- programming

command	Description
<code>b label</code>	Branch unconditionally to label
<code>:label</code>	Specify location of label for branch command
<code>N</code>	Add a new line to the pattern space and append next line of input into it.
<code>q</code>	Exit sed without processing any more commands or input lines
<code>t label</code>	Branch to label only if there was a successful substitution was made
<code>T label</code>	Branch to label only if there was no successful substitution was made
<code>w filename</code>	Write pattern space to filename
<code>x</code>	Exchange the contents of hold and pattern spaces

- `bash + sed`
 - Including sed inside shell script
 - heredoc feature
 - Use with other shell scripts on command line using pipe
- Working with `sed`
 - `sed -e "" edit.txt` -The default action of sed is to just print out the contents of the file if nothing is specified.
 - `sed -n` - the default action of printing is not performed
 - `sed -e '=' sample.txt` the = prints the line number
 - `sed -n -e '5p' sample.txt` -n says not to print anything else by default. 5p says to print the 5th line. Without -n all the lines will be printed and the 5th line will be printed 2 times. The fifth line is the address space
 - `sed -n -e '5!p' sample.txt` The ! means that all lines except the 5th line will be printed. ! Exclamation mark negates an address.
 - `sed -n -e '$!p' sample.txt` prints all except the last line. Careful with using ' ' instead of " " here.
 - `sed -n -e '5,8p' sample.txt` prints the 5th to the 8th line both inclusive.
 - `sed -n -e '=; 5,8p' sample.txt` prints all line numbers and from 5 to 8 prints lines also.
 - `sed -n -e '5,8{=;p}' sample.txt` prints line numbers for the line 5 to 8 alone.
 - `sed -n -e '1~2p' sample.txt` prints lines 1,3,5,7 ... Number coming after ~ specifies step size.
 - `sed -n -e '1~2!p' sample.txt` prints the remaining lines due to the negation specified by !
 - `sed -n -e '/microsoft/p' sample.txt` Supplying a phrase and an action. The phrase is microsoft and the action is to print every line containing the phrase.
 - `sed -n -e '/in place of/!p' sample.txt` prints the lines that do not contain the phrase "in place of"
 - `sed -n -e '/adobe/,+2p' sample.txt` prints the line containing "adobe" and two more lines that come immediately after that.
 - `sed -n -e '5d' sample.txt` deletes the 5th line and prints the rest
 - `sed -e '5,8d' sample.txt` deletes from the 5th to the 8th line and prints the rest
 - `sed -e '1,$d' sample.txt` deletes from the 1st to the last line and prints nothing
 - `sed -e '/microsoft/d' sample.txt` deletes all the lines containing microsoft and prints the rest
 - Most popular usage of the sed command is to substitute one phrase with another.
 - `sed -e 's/microsoft/MICROSOFT/g' sample.txt` search and replace. s implies search and g implies global.
 - `sed -e '1s/linux/LINUX/g' sample.txt` replaces 'linux' with 'LINUX' on only the first line.
 - `sed -e '1,$s/in place of/in lieu of/g' sample.txt` replaces 'in place of' with 'in lieu of' from the first line to the last line.
 - Modifying the incoming stream using the extended regular expression engine.
 - `sed -E -e '3,6s/^L[[:digit:]]+ //g' sample.txt` performs a search and replace from the 3rd to the 6th line of capital L followed by number/s and then a space. -E indicates that the Extended regular expression set should be used.
 - `sed -E -e '3,/symbolic/s/^L[[:digit:]]+ //g' sample.txt` performs a search and replace from the 3rd to the line where the phrase 'symbolic' occurs, of capital L followed by number/s and then a space. -E indicates that the Extended regular expression set should be used.
 - `sed -E -e '1~3s/^L[[:digit:]]+ //g' sample.txt` performs a search and replace from the 1st line every third line, of capital L followed by number/s and then a space. -E indicates that the Extended regular expression set should be used.
 - `sed -E -e '1~3!s/^L[[:digit:]]+ //g' sample.txt` Negation of the address range performs the opposite of the previous command.
 - Address range as a regular expression
 - `sed -E -e '/text/,/video/s/^L[[:digit:]]+ //g' sample.txt` performs a search and replace from the line that contains 'text' to the line where the phrase 'video' occurs, of capital L followed by number/s and then a space. -E indicates that the Extended regular expression set should be used.
 - `sed -e '1i -----header-----' -e '$a -----footer-----' sample.txt` Here i inserts before the first line and a appends after the last line.
 - `sed -e '/microsoft/i -----watchout-----' -e '/in place of/a -----alternative-----' sample.txt` 'watchout' appears above every line that contains microsoft and alternative comes below every line that contains 'in place of'
 - `sed -e '1~5i -----break-----' sample.txt` inserts 'break' after every 5 lines.
 - `sed -e '/microsoft/c -----censored-----' sample.txt` For every line that has 'microsoft' c or change command is executed
 - `sed -e '1~3c -----censored-----' sample.txt`
- An sed script file
 - more hf.sed

```
#!/usr/bin/sed -f
1i -----header-----
$a -----footer-----
1,5s/in place of/in lieu of/g
6i ----- simpler stuff here onward -----
6,$s/in place of.*/g
```

- First line mentions the interpreter
- last line removes all the characters whenever 'in place of' is encountered
- `sed -f hf.sed sample.txt` The `-f` implies that sed will use a file.
- `more clean.sed`

```
/[[[:alpha:]]{2}[[[:digit:]]{2}[[[:alpha:]]{2}[[[:digit:]]+]/!d
s/[ ]+/ /g
s/ ([[[:digit:]]+).*/ \1/g
```

- For the input file `block-ex-6.input` - File containing roll number and fees paid
- First line deletes all lines that dont contain roll number
- 2nd line replaces multiple spaces with single space
- 3rd line keeps number by back referencing
- `sed -E -f clean.sed block-ex-6.input`
 - Joining lines
- Example : joining lines which are ending with a `\`
- `cat join.sed`

```
#!/usr/bin/sed -f
:x /\$N
/\/s/\\n//g
/\/\$bx
```

- The `:` indicates a label. Whenever there is a `\` the `N` causes it to read the next line in the buffer.
- 2nd line - On the lines which have `\`, if there is a new line character it will be replaced with null.
- 3rd line on those lines which contain `\` we branch to the first line.
- `sed --debug -f join.sed sample-split.txt`
- The debug option helps to debug infinite loops in sed

L7.2

Version Control

- Every Save is effectively a new version of the code
 - "Make" - Compile only those parts of code that has changed. You do not touch what has not been modified.
 - If a group of programmers are working on a project with lots of codes and lots of files, following a modular approach (Each function as a separate file in C for example). There is a tacit understanding that programmers are not going to work on the same file.
 - Each programmer has multiple versions of the each file they worked on.
 - Why is version control necessary ? To trace back to a working version of code.
 - Versions will depend on number of users, number of files and number of versions. This needs to be kept in a database.
 - Two major version control systems
 - SVN - Centrally hosted and managed version system
 - Allows for one master who keeps track of the version of code that is being officially supported.
 - Storage Systems - Not if it fails but when it fails - When it fails no one can access. RAID - Redundant Array of Inexpensive/Independent Disks.
 - GIT - Distributed version control system
 - Even if something happens to the master server disappears nothing significant is lost because every collaborator has a copy of everything.
 - GIT system doesn't really require a server
- git
 - remote - server with which we synchronize
 - protocol for connection - git protocol - protocol by which we exchange information with remote and do version control.
 - options of using git
 - locally run git server
 - campus git server
 - gitlab
 - github.com
 - Two factor authentication for github

- app -> otp -> enter
 - app -> ask -> swipe
 - SMS -> OTP -> enter
 - customised for each repository/activity
 - personal access token
 - Activities
 - register on github.com
 - enable 2 factor authentication (Microsoft Authenticator App)
 - Create a repo
 - practice how to pull,push,git actions.
 - Developer Settings >> Personal Access Token
 - Create repository
 - `git clone url-of-github-rep.git`
 - append `.git` to url
 - folder will be created automatically
 - edit README.md using `vi README.md`
 - `git init` in the directory so that git understands that it is the same directory
 - creates a `.git` folder with all the paraphernalia that git requires
 - `ctrl + z` puts the program that was running as a background job. `kill %` kills the background job
 - `git remote add master url-of-github-rep`
 - it understands that there is a remote location that you have configured
 - `git config --global user.name "your_username_on_github"`
 - `git config --global user.email "your_github_registered_email"`
 - `git status` will check what is happening
 - `git add README.md`
 - `git commit -m "Message which describes what you have done"`
 - Use the personal access token created earlier
 - `vi ../pat` to store the token one level above the folder.
 - `git push` enter username and personal access token
-

L7.3

Github Brief Introduction

- Create account on Github
 - Configure 2 factor authentication and download recovery keys
 - Install Microsoft Authenticator App on your mobile
 - Login to github.com using TFA as a habit
- Creating your own repository
 - Create private repository
 - Get a personal access token to use this
 - Clone the repository on your computer `git clone url_of_the_repo`
 - Configure the folder for git using `git init`
 - Tell git about yourself: `git config`
 - `git config --global user.name "your_username_on_github"`
 - `git config --global user.email "your_github_registered_email"`
 - Configure the remote `git remote add master url_of_the_repo`
 - Change some files if you wish
 - Run the `git status` command to understand what is going on.
 - Stage them to be ready to send to remote using `git add modified_filename`
 - `git add .` will push everything to the server
 - Commit the change using `git commit -m "message"`
 - Type `git status` again
 - Push the changes using `git push`
- Working with branches
 - Create a new branch for a repository you are already working on - `git branch -git branch "Panda"`
 - Check out the branch
 - `git checkout`
 - `git checkout Panda`
 - `git status` shows that you are on the Panda branch
 - Make some changes to some files
 - All changes are now to the branch
 - `git add README.md`
 - `git commit -m "This is from my PC"`
 - `git push --set-upstream origin Panda`
 - Merge the branch with the master/main
 - `git checkout main`
 - `git merge Panda`
 - On the website 'Compare and Pull Request'. Then 'Create Pull Request'. Then 'Merge Pull Request'
 - Remote checks if there is any conflict. Creating and merging branches is part of the coding cycle.
- Contributing to others' repositories
 - Fork their repository
 - create your branch

- make some changes to your branch and push those to the server
- on the remote server, compare and create a pull request
- Allowing contributors to chip in
 - Look at pull requests and approve them
 - Resolve and conflicts in some files

Week 8

Knowing your hardware

Hardware items

- CPU
- Storage & Partitions
- Graphics Card
- Memory Modules
- Battery & status
- Network devices & configuration

Packages to install

- `clinfo`
- `coreutils`
- `dmidecode`
- `fdisk`
- `hardinfo`
- `hdparm`
- `hwinfo`
 - Will probe hardware and show output on the screen. Information about PCI, partitions, keyboard, tablet, earphones etc..
- `lshw`
 - List hardware. Output is in various sections. You can choose a particular section eg : `lshw -c display` or `lshw -c processor` or `lshw -c memory`
- `memtester`
- `net-tools`
- `pciutils`
- `procp`
- `sysstat`
- `upower`
- `util-linux`
- Demo
 - `cat /proc/cpuinfo` gives information about the CPU.
 - `cat /proc/partitions` gives partition information. The loop partitions are meant for snap packages
 - `lsblk -o NAME,SIZE` gives information about the number of block devices that are available.
 - `lspci` gives the list of PCI devices connected to the computer using the PCI bus.
 - `free` gives details about the amount of memory used. It is a practice to have double the size of the memory as swap
 - `sudo dmidecode --type memory` gives information about the memory and modules.
 - `hardinfo` is a GUI utility
 - `clinfo` gives information about the graphics card.
 - `upower -e` to know about the battery status. This will give a list. Choose the one that says battery and execute `upower -i /org/freedesktop/UPower/devices/battery_BAT0` for example.
 - `sudo hdparm -Tt /dev/sda` runs diagnostics on the ssd or hdd. Timing cached reads and buffered disk reads.
 - `iostat -dx /dev/sdb` gives information about speeds of various disks.
 - `ifconfig` is a network utility that gives information about ethernet / loopback / wifi adapter.

Prompt strings

- Context for prompt strings
 - `bash`, `dash`, `zsh`, `ksh`, `csk`
 - `python`
 - `octave` - Matlab compatible numerical package
 - `gnuplot` - Plotting tool
 - `sage` - symbolic computing package. Perhaps better than Mathematica
- bash prompts
 - PS1 : primary prompt string : `$`
 - PS2 : secondary prompt for multi-line input : `>`
 - PS3 : prompt string in select loops : `#?`
 - PS4 : prompt string for execution trace : `+` Explanation: There are 4 bash prompts that are configured. What we see is normally the primary prompt when we open the shell. PS2 is shown when a command is incomplete. PS3 is shown when we run a bash script in a select loop. PS4 is shown when every command that is executed is displayed on the screen - when we use the option `set -x`
- Escape sequences

-	Description
\A	Current time in 24-hour as hh:mm
\u	Current user's username
\d	Date in "weekday month day" format
\w	Current directory
\h	Hostname upto first period
\W	Basename of current directory
\H	Complete hostname
\#	Current command number
\s	Name of the shell
\\$	If uid is 0, # else \$
\t	Current time in 24-hour as hh:mm:ss
\@	Current time in 12-hour a.m/p.m
\T	Current time in 12-hour as hh:mm:ss
\\	A literal \ character

\u@\h:\w\\$ - This is the default value of PS1. Username @ machine name : current dir \$ if user is not superuser

To change what is displayed in the prompt string.

- Python command line
 - ps1 and ps2 are defined in the module sys
 - Change sys.ps1 and sys.ps2 if needed
 - Override __str__ method to have dynamic prompt >>> Default python command prompt

Demo - echo \$PS1 - gives \[e]0;\u@\h: \w\a\\${debian_chroot:+(\$debian_chroot)}\[033[01;32m\]\u@\h\[033[00m\]:\[033[01;34m\]\w\[033[00m\]\\$ - It can be changed PS1="\u@\h:\w\\$ " . The color will be lost from above prompt string. - If you do source .bashrc you get back the prompt after messing up the prompt string. - less .bashrc to see where it is defined. - echo \$PS3 doesn't display anything. - select x in alpha beta gamma; do echo \$x; done displays PS3 - In octave x=[1:1:100] creates array

- In SageMath `plot(sin(x),x,0,2*pi)`

Important Utilities

- find – locating files and processing them
- tar, gzip etc – packaging collections of files
- make – conditional actions

find

- find [pathnames] [conditions]

Condition	Description
-name	pattern to match filenames
-type	File type code eg., c for character file, d for directory, l for symbolic link etc.,
-atime	Files accessed +n (more than n), -n (less than n) days ago
-ctime	File changed +n (more than n), -n (less than n) days ago

Condition	Description
<code>-regex</code>	Regular expression for pattern of filenames. Combine with <code>-regextype posix-basic</code> , <code>posix-egrep</code> etc.,
<code>-exec</code>	Command to run using <code>{}</code> as place holder for filename
<code>-print</code>	Print the full path name of matching files

- Examples

- Find
 - `find $HOME -print | wc -l` will print the number of files in the home directory
 - `find $HOME -mtime -2 -print` prints files modified in the last 2 days
 - `find . -mtime -2 -print` prints files modified in the last 2 days in the current working directory
 - `find /usr -type d -name 'man*' -print` directories beginning with the word man in /usr directory
 - `find . -size +10M -print` finds files that are more than 10 MB
 - `find . -size +10M -print -exec ls -lsh {} \;` every file that is greater than 10 MB will be inserted in the curly braces. The output will be file name followed by long listing.
 - `find . -name '*.jpg' -exec -ls -sh {} \;` lists all the jpeg files and their size
- Compression
 - `cp -r /var/log logfiles` To illustrate compression capability get a copy of the log folders
 - `du -sh` shows the size it occupies
 - `ls -lR` displays recursively
 - `tar -cvf logfiles.tar logfiles/` bundles all the files in the directory as logfiles.tar
 - Compress it using `gzip logfiles.tar`
 - `bzip2 logfiles.tar` shrinks it better but takes some more time
 - `bzip2 -d logfiles.tar.bz2` unzipping the files
 - `compress logfiles.tar` is very fast but compression is not much
 - For a directory that has lot of subdirectories and lot of small files, it is a good idea to package it as a tar file
 - `rm -rf logfiles` to remove the original logfiles directory
 - `uncompress logfiles.tar.Z` unzips the tar
 - `tar -xvf logfiles.tar` gives back the directory
- Make
 - `make.file` is given below

```

BACKUPDIR=/home/username/backup
DATESTAMP=$(shell date+%Y-%m-%d %G.%M")
TARBALL=backup_$(DATESTAMP).tar
backup: file1 file2
    @echo "Backing up file1 file2 to $(DATESTAMP)"
    tar -cvf $(TARBALL) file1 file2
    @ls -l $(TARBALL)
    mv $(TARBALL) $(BACKUPDIR)

```

- The `:` in front of `=` in the `DATESTAMP` line indicates one-time substitution.
- `make -f make.file backup`
- The action is performed only if one of the dependencies has changed. It will get executed again only if file1 or file 2 has changed.
- Any maintenance activity can be taken up as a make file command.

file packaging

- Deep file hierarchies
- Large number of tiny files
- `tar` : collect a file hierarchy into a single file
- `gzip` : compress a file
- Applications: backup, file sharing, reduce disc utilization Explanations:
- Sometimes when there are several small files in a hierarchy structure, the files may occupy the minimum block size so there is a wastage of space. In such situations doing a `tar` will save space.

Possibilities

- `tar`, `zip`
- `compress` (`ncompress`), `gzip` (`ncompress`), `bzip2` (`bzip2`), `xz` (`xz-utils`), `7z` (`p7zip-full`)
- Tarballs like `bundle.tgz` for package + compress
- Time & memory required to shrink / expand versus size ratio
- Portability
- Unique names using timestamp, process ID etc., for backup tarballs Explanation :
- Plain text or ASCII files can be compressed to a very good ratio (almost 1:10) if the file contains repeating patterns.
- For more efficiency, first zip and then make a tar. Zipping the files while adding it to tar - `.tgz` file format combines tar and gzip together.
- The decision on which method to use is taken based on time required, space occupied etc..

make

- `make -f make.file`

```
# comments
TMP_FILES = *.o *.aux
.PHONY : clean
target : prerequisites
recipe $(OPTION_NAME)
clean:
rm -f $(TMP_FILES)
```

Network & ssh

Accessing remote machines on command line

- IPv4 address range
 - Localhost
 - `127.0.0.0/8`
 - Private network
 - Class A : `10.0.0.0/8` - 16,777,216
 - Class B : `172.16.0.0/12` - 1,048,576
 - Class C : `192.168.0.0/16` - 65,536
 - Public network
- Explanations
 - Interfaces between Private and Public network is a gateway
 - Local host address is the way by which the system can refer to itself.
 - The number after the slash indicates how many IP addresses are possible.
 - Class A : The three parts following 10 can be any number between 0 and 255. $2^{24}-2=16,777,216$. This is useful for a large organisation
 - You can create orivate networks within private networks upto 3 levels
- Ways to gain remote access
 - VPN access
 - ssh tunneling
 - secure shell tunneling
 - Remote desktop : x2go, rdp, pcoip,
 - rdp is popular among Windows machine
 - pcoip with VMWare
 - Desktop over browser: Apache Guacamole
 - Commercial, over internet : Teamviewer, AnyDesk,Zoho assist, ...
- Some important ports

Port	Service	Description
21	ftp	File transfer
22	ssh	Secure Shell
25	smtp	Simple Mail Transfer Protocol
80	http	Hypertext Tranfer Protocol
443	https	Secure Hypertext Tranfer Protocol
631	cups	Common Unix Printing System
3306	mysql	MySQL database

- Firewall
 - Ports open on my machine
 - Ports needed to be accessed on remote machine
 - Network routing over the port
 - Firewall controls at each hop
- Protecting a server
 - Server with a public service > Web Application Filter > Network Firewall > Anonymous users
- SELinux
 - Security Enhanced Linux mode available on Ubuntu too, apart from server grade flavors like CentOS, Fedora, RHEL, SuSE Linux etc.,
 - Additional layer of access control on files to services
 - A layer beyond file permissions.
 - Role Based Access Control

- Process sandboxing, least privilege access for subjects
- Every process will have restriction of reading the memory locations of other processes
- Check using `ls -lZ` and `ps -eZ`
- RBAC items: `user (unconfined_u)`, `role (object_r)`, `type (user_home_t)`, `level (s0)`
- Role Based Access Control
- Modes: `disabled`, `enforcing`, `permissive`
- Tools: `semanage`, `restorecon`
- SELinux is recommended for all publicly visible servers

- Network tools

Tool	Description
<code>ping</code>	To see if the remote machine is up
<code>traceroute</code>	Diagnostics the hop timings to the remote machine
<code>nslookup</code>	Ask for conversion of IP address to name
<code>dig</code>	DNS lookup utility
<code>netstat</code>	Print network connections
<code>mxtoolbox.com</code>	For help with accessibility from public network
<code>whois lookup</code>	Who owns which domain name
<code>nmap</code>	(careful !) Network port scanner
<code>wireshark</code>	(careful !) Network protocol analyzer. Sniff packets through the network for diagnostics.

- High Performance Computing
 - Look at www.top500.org for statistics
 - All 500 HPC machines are GNU Linux based.
 - Accessing a remote HPC machine is usually over SSH
 - Long duration jobs are submitted to a job scheduler for execution
 - Raw data if large needs to be processed remotely before being transferred to your machine (network charges? bandwidth?)
 - Comfort with command line is a must
- Demo
 - `ifconfig` shows a Class A IP range network within the campus
 - `nslookup www.iitm.ac.in` shows it is connected to `waf6.iitm.ac.in` web application filter.
 - `dig -x 142.250.206.164`
 - `aqua.iitm.ac.in` HPC cluster is not visible from the internet.
 - `qstats` is used to get job status

- `netstat` to display active connections.

Automating scripts

Scheduled, recurring, automatic execution of scripts

- `cron`
 - Service to run scripts automatically at scheduled times
 - Tools: `at`, `crontab`, `anacron`, `logrotate`
 - Script locations:
 - `/etc/crontab`
 - `/etc/cron.d`
 - `/etc/cron.hourly`
 - `/etc/cron.daily`
 - `/etc/cron.weekly`
 - `/etc/cron.monthly`

Example of job definition:

```
.----- minute (0 - 59)
| .----- hour (0 - 23)
| | .----- day of month (1 - 31)
| | | .----- month (1 - 12) OR jan,feb,mar,apr ...
| | | | .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
| | | | |
* * * * * user-name command to be executed
```

- Job definition
 - 5 2 * * 1-5 root cd /home/scripts/backup && ./mkbackup.sh
 - 5 minute (0-59)
 - 2 hour (0-23)
 - * day of the month (1-31)
 - * month (1-12) or jan, feb, ...
 - 1-5 day of week (0-6) or sun,mon, ...
 - root user-name
 - cd /home/scripts/backup command
 - The above command runs mkbackup.sh as root every working day at 02:05 AM
- Demonstration
 - The first time crontab is used you have to select the default editor
 - crontab is in etc directory
 - anacron is run by the System administrator
 - cron.daily is a folder in etc that displays daily tasks. Similarly cron.hourly, cron.monthly, cron.weekly
 - By placing a script in any of these directories you can make it run at the specified schedule
 - By running crontab -e you can execute a specific script at a time. Customize timely running of scripts.
- Startup scripts
 - Startup scripts: /etc/init/, /etc/init.d/
 - Runlevel scripts:

Level	Location	Description
0	/etc/rc0.d/	Shutdown and power off
1	/etc/rc1.d/	Single user mode
2	/etc/rc2.d/	Non GUI multi-user mode w/o networking
3	/etc/rc3.d/	Non GUI multi-user mode with networking
4	/etc/rc4.d/	Non GUI multi-user mode for special purposes
5	/etc/rc5.d/	GUI multi-user mode with networking
6	/etc/rc6.d/	Shutdown and reboot

Managing Storage

LVM & RAID

- LVM
 - Logical Volume Management
 - Pooling multiple storage devices as a single logical volume
 - lvm2 tools : create and manage virtual block devices from physical devices
 - Suppose you need a very large partition but there is no HDD available of that size, you can define a logical volume that spans over multiple HDDs.
 - Logical Volumes are mounted by the GNU Linux OS, which are mapped over multiple physical disks.
- RAID
 - Redundant Arrays of Independent Disks
 - Distributing data over multiple discs for redundancy / speed / increased capacity
 - Raid Controller : software or hardware
- RAID modes
 - usable capacity < actual capacity

RAID Mode	Min drives	Description	Comments
RAID 0	2	Striping	Speed up
RAID 1	2	Mirroring	Read is n times faster, n-1 drive failures tolerated
RAID 5	3	Block-level striping with distributed parity	1 drive failure tolerated, Read is n times faster, write is n-1 times faster
RAID 6	4	Block-level striping with dual distributed parity	2 drive failures tolerated, read is n times faster, write is n-2 times faster

- Explanation

- RAID 0 - You are using 2 disks as 1. Half of one file is stored on 2 disks. Doubles speed of access of a file. Write Speed is 2x and Read Speed is 2x for 2 disks. If there are n disks in RAID 0 equivalent storage is size of minimum disk * n.
- RAID 1 - Any piece of the file is written to both the disks. Reading is 2x but writing is n-1. People tend to use RAID 1 for OS alone.
- RAID 5 - When you have more than 3 disks. Data is written to more than one disk. If one fails nothing is lost.
- RAID 6 - Parity over 2 disks. If 2 disks fail you still have all your data.
- Most of the hardware supports hot-swap.
- Useable capacity is less than the actual capacity
- For storage people use RAID 5 or RAID 6.
- Demo
 - `df -h` to check system storage
 - Which RAID configuration to use to improve read performance and sustain at least one disk failure without losing data ? RAID 4, RAID 6.

Images

